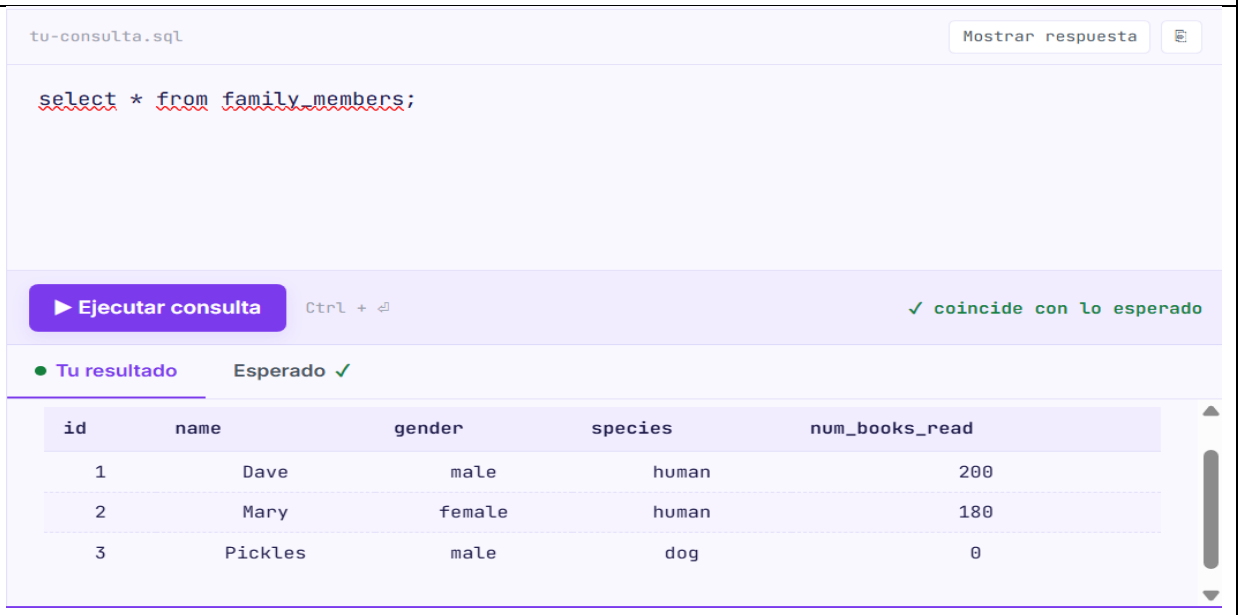
SQL & WEB SECURITY CHALLENGE

ENTREGABLE 3 Y 4

Dana Paola Martínez Hernández
HACKER WOMEN COUNCIL
17/10/2026

SQL EASY: DOCUMENTACION DE LAS 31 CONSULTAS

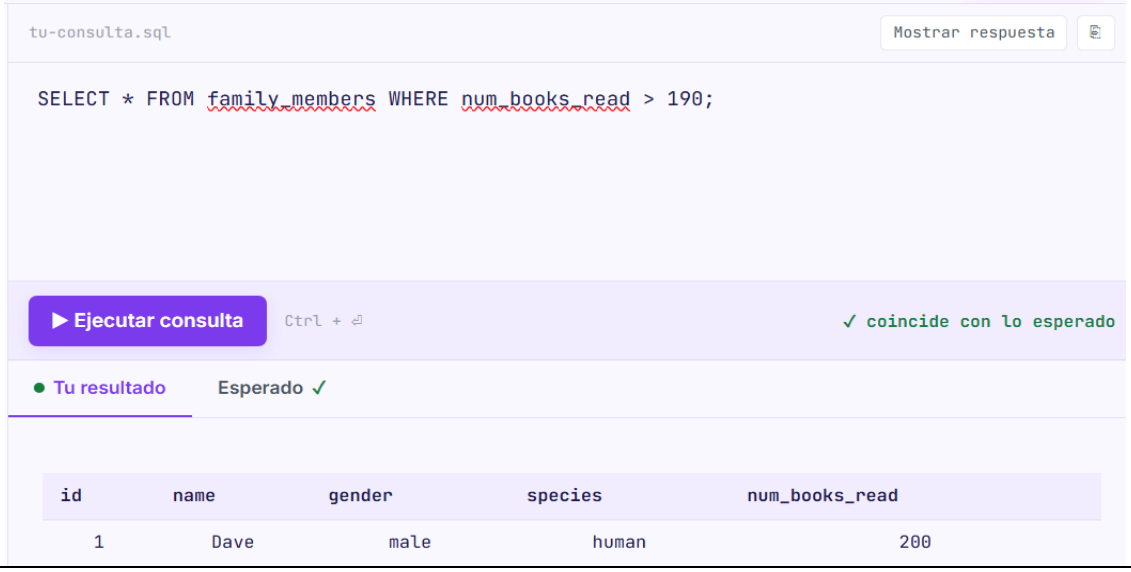
Número y nombre del ejercicio	**Ejercicio 1 - SELECT ***
Objetivo del ejercicio	Tenemos una tabla llamada **family_members,** de ella tenemos que obtener todos los datos que contenga.
Concepto SQL	Buscamos aprender a utilizar el *, que significa que todas las columnas serán devueltas.
Consulta ejecutada	**select * from family_members;**
Resultado	
Interpretación	Los datos obtenidos de esta consulta SQL, aplicada a la tabla **family_members** son las columnas: *id*, *name*, *gender*, *species* y *num_books_read*, junto con su respectiva información.
Aprendizaje	Aprendí que si utilizo el * en una consulta SQL puedo hacer que la pagina me de todo lo que tiene almacenado, sin necesidad de poner uno por uno.
Aplicación en el mundo real	Es muy útil cuando apenas estás explorando una base de datos nueva y quieres ver la estructura completa de una tabla con todas sus columnas, datos y registros.

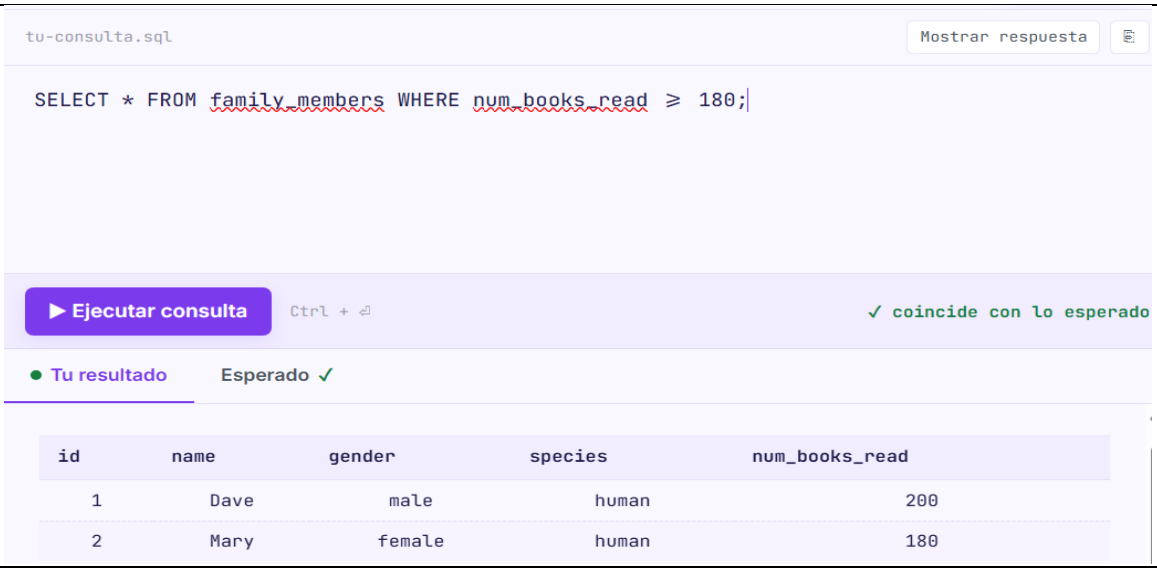
Número y nombre del ejercicio	**Ejercicio 2 - SELECT columnas específicas**
Objetivo del ejercicio	De la tabla **family_members** teníamos que obtener los datos solo de las columnas ***name y species.***
Concepto SQL	Aprender a utilizar el comando ***select*** para seleccionar y obtener información específica.
Consulta ejecutada	**select name, species from family_members;**
Resultado	
Interpretación	La información obtenida fue el contenido de las columnas ***name y species***, que muestran los nombres de los usuarios y su especie.
Aprendizaje	Aprendí a seleccionar información, pues es más práctico utilizar el comando ***select*** para cuando busco información especifica que utilizar el * lo que me mostrara información de más que no necesito.
Aplicación en el mundo real	Sirve mucho en sistemas o aplicaciones donde solo necesitas mostrar datos puntuales, por ejemplo, cuando en una lista de contactos solo quieres desplegar el nombre y el correo electrónico sin cargar toda la información del usuario.

Número y nombre del ejercicio	**Ejercicio 3 – WHERE …Equals**
Objetivo del ejercicio	Ejecutar una consulta para obtener información de la tabla **family_members**, columna **species,** pero solo las filas correspondientes a **dog.**
Concepto SQL	Aprender a utilizar la cláusula **WHERE** para seleccionar la información de una fila en particular
Consulta ejecutada	**Select * from family_members where species = 'dog';**

Resultado	
Interpretación	El resultado de la consulta mostro las filas que hacían referencia a **dog**, dentro de la columna **species**, en este caso solo hubo una sola fila.
Aprendizaje	Aprendí como obtener información especifica dentro de las columnas, de igual manera para ser más práctica al buscar solo cierta información sin tener que consultar toda la tabla.
Aplicación en el mundo real	Se utiliza todo el tiempo en sistemas de autenticación y búsquedas, por ejemplo, cuando en un e-commerce buscas productos filtrando por una categoría específica como “laptops”.

Número y nombre del ejercicio	**Ejercicio 4 – WHERE …Mayor que**
Objetivo del ejercicio	Ejecutar una consulta para obtener información de la tabla **family_members,** específicamente todas las filas de miembros de la familia cuyo **num_book_read** sea mayor que 190.
Concepto SQL	Aprender a utilizar la cláusula **WHERE** junto con los signos **< >** para cuando buscamos datos basados en un campo numérico.
Consulta ejecutada	**SELECT * FROM family_members WHERE num_books_read > 190;**
Interpretación	El resultado de la consulta mostro las filas que contenían un numero **> que 190** dentro de la columna **num_book_read,** en este caso solo fue una fila
Aprendizaje	Aprendí a filtrar información utilizando datos numéricos y operadores de comparación como >, lo que me permite buscar solo los registros que superen cierta cantidad sin tener que revisar la tabla manualmente.
Aplicación en el mundo real	Tiene muchas aplicaciones, por ejemplo, en una tienda en línea para filtrar los productos que cuesten más de cierto precio.

Resultado	

Número y nombre del ejercicio	**Ejercicio 5 – WHERE …Mayor o igual que**
Objetivo del ejercicio	Obtener todas las filas de **family_members** que tengan un valor mayor o igual a 180, pero que estén dentro de la columna de **num_books_read.**
Concepto SQL	En este ejercicio aprendemos a ejecutar una consulta utilizando **WHERE,** pero con diferentes operadores de comparación como: = “igual a”, > “mayor que”, < “menor que”, >= “mayor o igual que” y <= “menor o igual que”. De igual manera redactamos el comando por nuestra cuenta sin ayuda o pistas dentro de las instrucciones del ejercicio.
Consulta ejecutada	**SELECT * FROM family_members WHERE num_books_read >= 180;**
Resultado	
Interpretación	El resultado de la consulta muestra 2 filas de datos que contenían un número igual o mayor que 180 dentro de la columna **num_books_read.**

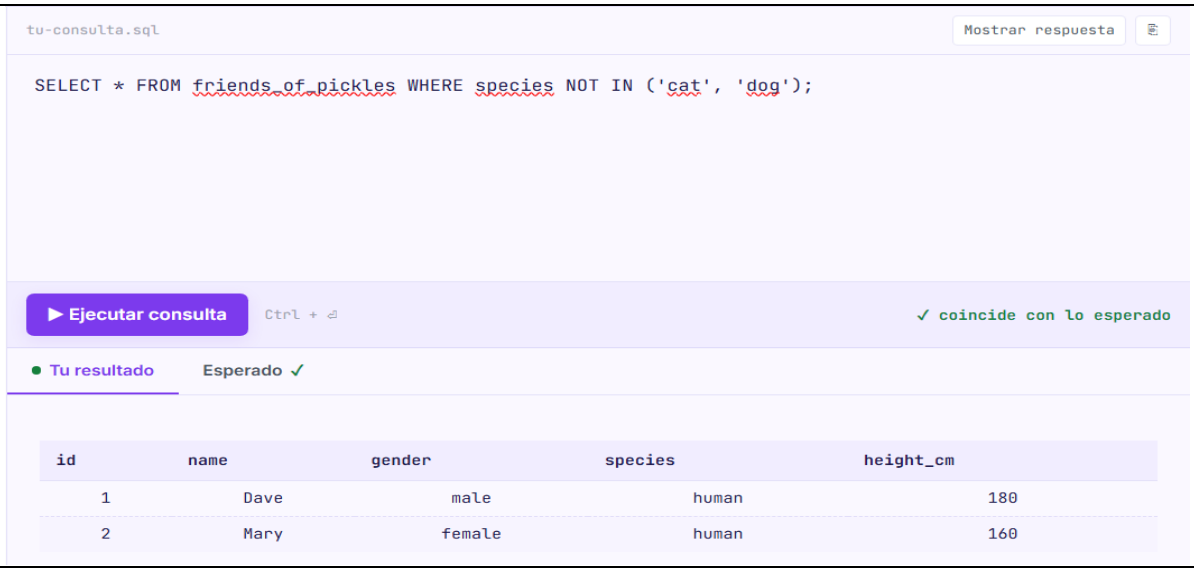
Aprendizaje	Aprendí a utilizar el operador de comparación mayor o igual que (>=) dentro de la cláusula **WHERE**, lo que me permite incluir también el límite exacto de la condición, en este caso el 180, al momento de filtrar datos numéricos.
Aplicación en el mundo real	Sirve mucho cuando necesitas incluir un límite exacto; por ejemplo, en una escuela para buscar a los alumnos que sacaron una calificación de 80 o más.

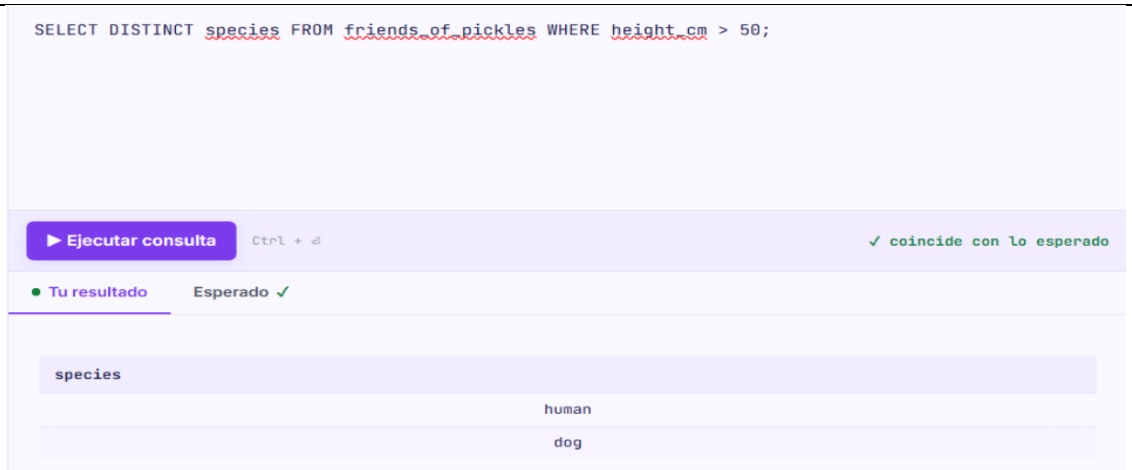
Número y nombre del ejercicio	**Ejercicio 6 - AND**
Objetivo del ejercicio	Obtener todos los amigos de Pickles que son dogs y miden menos de 45cm de altura, dentro de la tabla llamada ***friends_of_pickles,*** en las columnas de ***height_cm*** y ***species.***
Concepto SQL	Nos enseña a ejecutar una consulta utilizando la palabra clave **AND,** pues con ella podemos buscar múltiples atributos cuando la acompañamos con la cláusula **WHERE.**
Consulta ejecutada	**SELECT * FROM friends_of_pickles WHERE height_cm < 45 AND species = 'dog';**
Resultado	
Interpretación	El resultado de la consulta muestra la tabla **friends_of_pickles,** donde nos muestra las filas 5 y 6, pues son las que contienen los datos filtrados de amigos que son dogs y miden menos de 45cm.
Aprendizaje	Aprendí a usar la palabra clave AND para juntar dos condiciones al mismo tiempo en una sola consulta, lo que me ayuda a filtrar los resultados de forma más exacta cuando necesito que se cumplan varias reglas a la vez.
Aplicación en el mundo real	Sirve mucho para cuando haces búsquedas avanzadas en plataformas, por ejemplo, en una tienda en línea para buscar productos que cuesten menos de cierto precio y que sean de una marca en específica.

Número y nombre del ejercicio	**Ejercicio 7 - OR**
Objetivo del ejercicio	Realizar una consulta donde el resultado me muestre todos los amigos de Pickles que son dogs o que miden menos de 45 cm de altura
Concepto SQL	Con este ejercicio aprendemos que, si en una consulta utilizamos la cláusula ***WHERE*** junto a un operador lógico como ***OR***, podemos buscar filas que coincidan con múltiples atributos.
Consulta ejecutada	**SELECT * FROM friends_of_pickles WHERE height_cm < 45 OR species = 'dog';**
Resultado	
Interpretación	El resultado de la consulta realizada fue lo esperado, pues nos muestra 5 filas con los datos de los amigos de Pickles que son dogs o que miden menos de 45 cm.
Aprendizaje	Comprendí cómo utilizar el operador lógico **OR** junto con la cláusula **WHERE**. Aprendí que este operador es muy útil cuando queremos ampliar los resultados de una búsqueda, ya que basta con que se cumpla **al menos una** de las condiciones establecidas para que el registro sea seleccionado.
Aplicación en el mundo real	Se usa en los filtros de cualquier app; por ejemplo, cuando en Netflix buscamos películas de **'Comedia'** o que estén **'Recién agregadas'**, para que el sistema te muestre un catálogo más grande según lo que uno quiera ver.

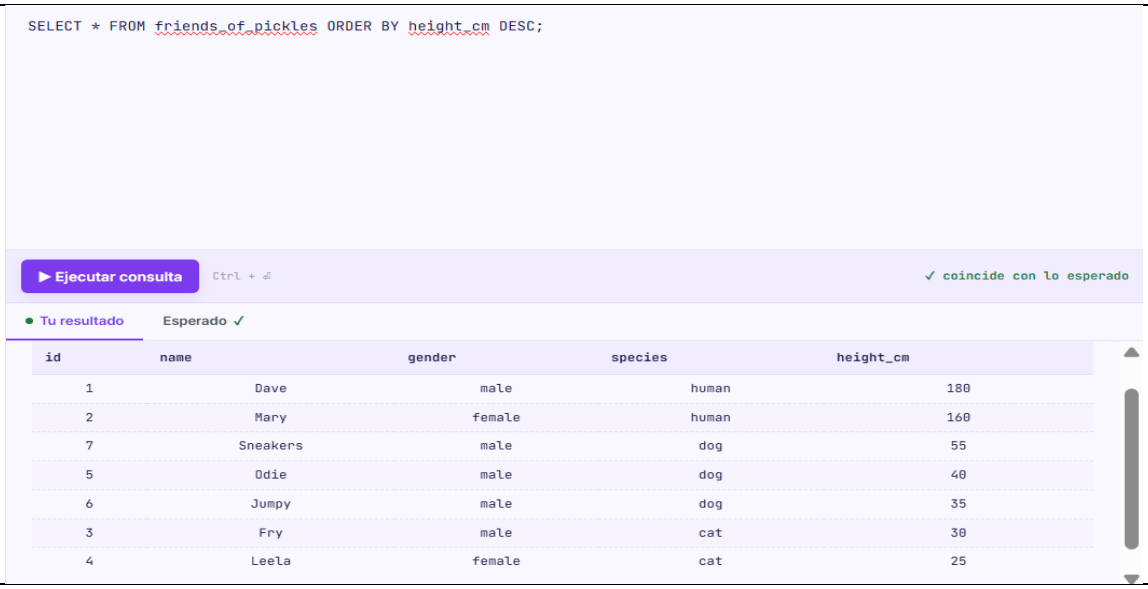
id	name	gender	species	height_cm
3	Fry	male	cat	30
4	Leela	female	cat	25
5	Odie	male	dog	40
6	Jumpy	male	dog	35
7	Sneakers	male	dog	55

Número y nombre del ejercicio	**Ejercicio 8 – IN**
Objetivo del ejercicio	Ejecutar una consulta para obtener todas las filas de la tabla **friends_of_pickles** excluyendo a los **dogs** y a los **cats**, utilizando el operador condicional para filtrar los datos que no coincidan con esos valores específicos.
Concepto SQL	Nos enseña que al utilizar la cláusula **WHERE** acompañada de **NOT IN**, podemos excluir de una consulta múltiples valores específicos a la vez (como 'cat' o 'dog'), evitando tener que escribir varias condiciones separadas con **AND** u operadores de desigualdad.
Consulta ejecutada	**SELECT * FROM friends_of_pickles WHERE species NOT IN ('cat', 'dog');**

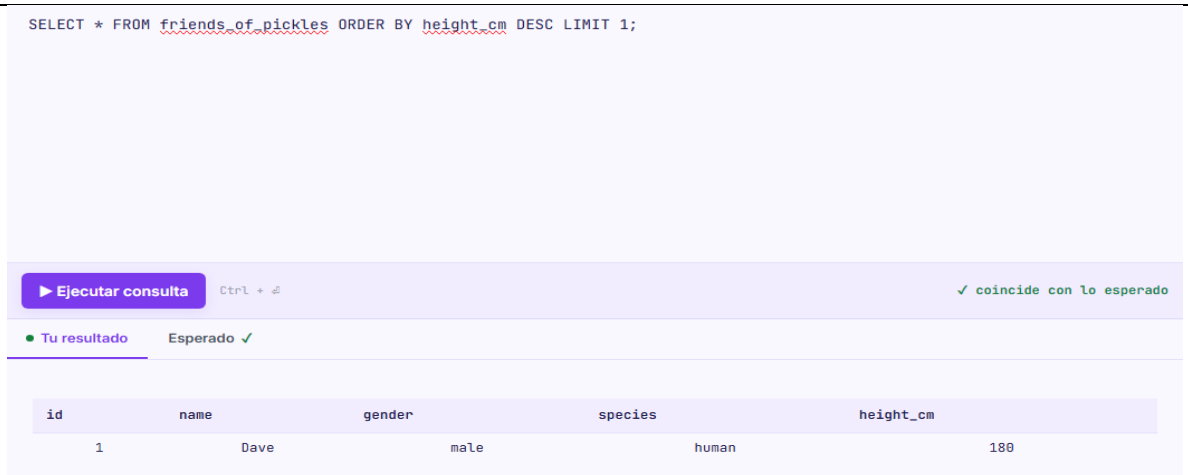
Resultado	
Interpretación	El resultado de la consulta muestra únicamente las filas donde la especie no es cat ni dog (en este caso, los registros de humanos), trayendo toda su información respectiva de la tabla.
Aprendizaje	Aprendí a usar el operador **NOT IN** para descartar listas de varios elementos de un solo golpe, lo que hace que las consultas sean más limpias y prácticas cuando necesitas omitir ciertos datos específicos.
Aplicación en el mundo real	Es muy útil en sistemas reales; por ejemplo, en una base de datos de usuarios para buscar cuentas activas excluyendo aquellos roles que estén suspendidos o bloqueados.

Número y nombre del ejercicio	**Ejercicio 9 - DISTINCT**
Objetivo del ejercicio	Ejecutar una consulta para obtener una lista de las especies de animales distintas que tengan una altura mayor a 50 cm.
Concepto SQL	Nos enseña a utilizar la palabra clave ***DISTINCT*** dentro de una consulta para limpiar los resultados y asegurarnos de que la información devuelta muestre únicamente valores únicos, eliminando las filas repetidas cuando un mismo dato aparece varias veces en la tabla.
Consulta ejecutada	**SELECT DISTINCT species FROM Friends_of_pickles WHERE height_cm >50;**
Resultado	

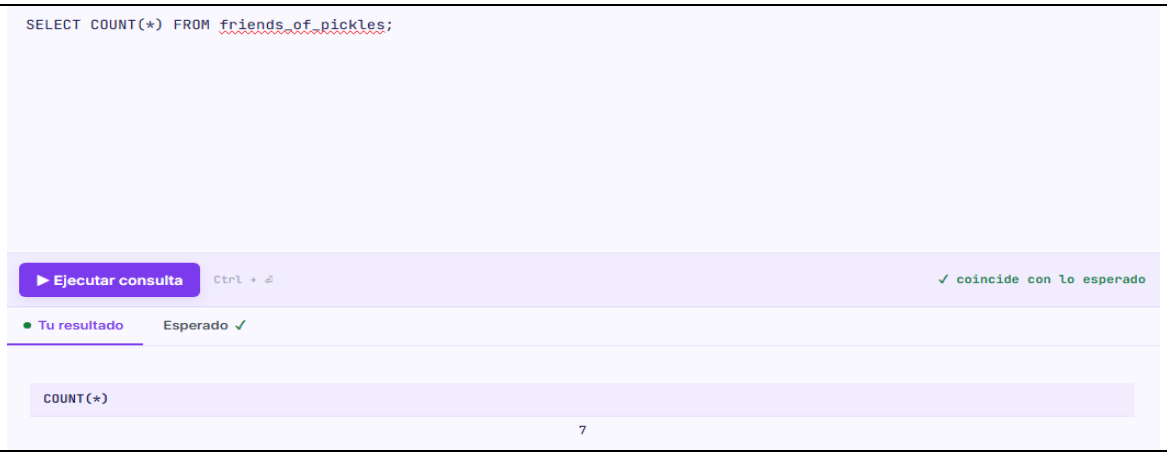
Interpretación	El resultado de la consulta muestra de forma exitosa los datos solicitados, otorgándonos una lista limpia con las especies diferentes de animales que cumplen con la condición de altura sin duplicados.
Aprendizaje	Aprendí a usar ***DISTINCT*** para pedirle a la base de datos que limpie los resultados y no me repita el mismo dato varias veces, mostrándome solo los valores únicos de una columna.
Aplicación en el mundo real	Es muy útil en sistemas reales; por ejemplo, cuando entras a una tienda en línea y quieres ver un menú desplegable con las marcas y sus productos disponibles, sin que el nombre de la marca se repita cientos de veces por cada artículo que pertenece a ella.

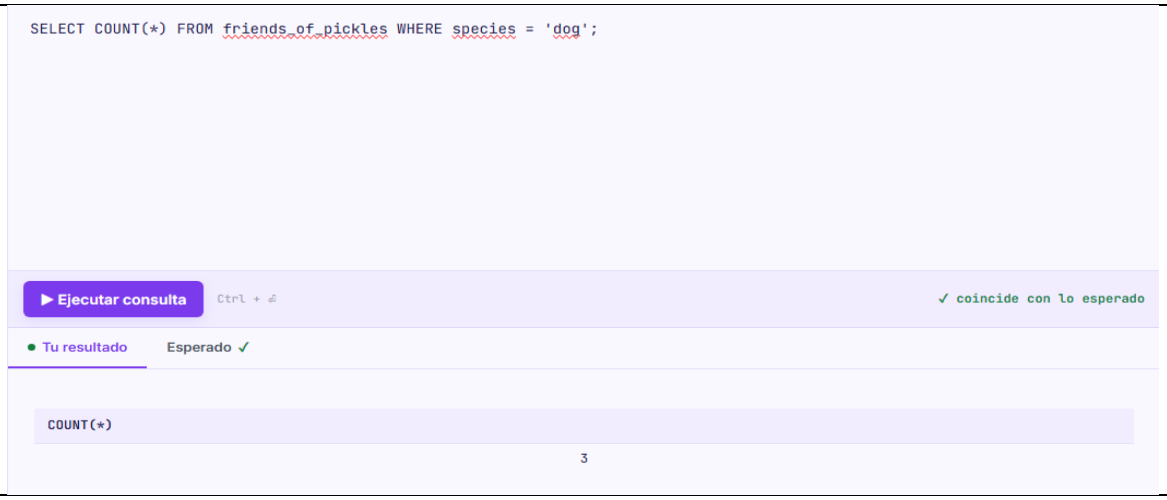
Número y nombre del ejercicio	**Ejercicio 10 – ORDER BY**
Objetivo del ejercicio	Ejecutar una consulta para ordenar los registros de la tabla ***friends_of_pickles*** de forma descendente basándonos en los datos de la columna ***height_cm***, es decir, acomodándolos desde el valor más alto hasta el más pequeño.
Concepto SQL	Nos enseña a utilizar la cláusula ***ORDER BY*** para organizar el resultado de una consulta según los valores de una columna específica. Por defecto los ordena de menor a mayor, pero si agregamos la palabra clave ***DESC*** al final, logramos hacer que se muestren en orden descendente.
Consulta ejecutada	**SELECT * FROM Friends_of_pickles ORDER BY height_cm DESC;**
Resultado	
Interpretación	El resultado muestra de manera exitosa todas las filas de la tabla ordenadas por completo, mostrando la información desde el registro que tiene mayor altura hasta llegar al de menor estatura según la columna indicada.
Aprendizaje	Aprendí a usar la cláusula ***ORDER BY*** junto con el modificador ***DESC*** para controlar la dirección en la que se acomodan los datos devueltos, permitiéndome presentar la información organizada de mayor a menor cuando lo necesité.
Aplicación en el mundo real	Es muy útil en sistemas donde se necesita jerarquizar información; por ejemplo, en una escuela para ordenar una lista de alumnos por su calificación final de la más alta a la más baja, o en una tienda en línea para mostrar los productos ordenados del precio más caro al más económico.

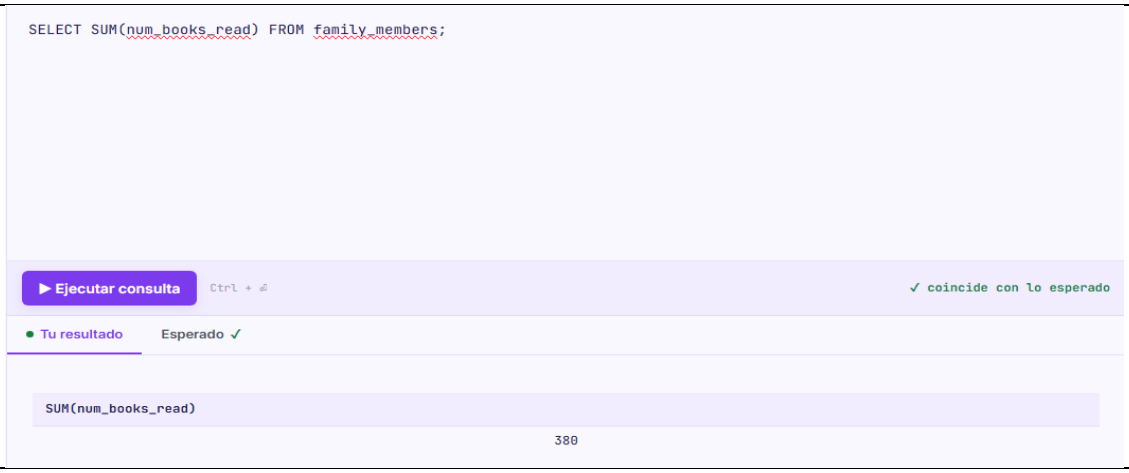
id	name	gender	species	height_cm
1	Dave	male	human	180
2	Mary	female	human	160
7	Sneakers	male	dog	55
5	Odie	male	dog	40
6	Jumpy	male	dog	35
3	Fry	male	cat	30
4	Leela	female	cat	25

Número y nombre del ejercicio	**Ejercicio 11 -LIMIT # de filas devueltas**
Objetivo del ejercicio	Ejecutar una consulta para obtener todas las columnas de la tabla ***friends_of_pickles,*** pero limitando el resultado para devolver únicamente la fila del registro del más alto, de acuerdo con la columna de ***height_cm.***
Concepto SQL	Nos enseña a utilizar la cláusula ***LIMIT***, la cual sirve para restringir el número de filas que nos devuelve una consulta. Cuando se combina con ***ORDER BY DESC***, primero ordena los datos del más alto al más bajo y después ***LIMIT*** recorta el resultado para quedarse solamente con el número de filas señaladas en la consulta.
Consulta ejecutada	**SELECT * FROM Friends_of _pickles ORDER BY height_cm DESC LIMIT 1;**
Resultado	
Interpretación	El resultado de la consulta muestra de forma exitosa únicamente una fila con todos los datos y columnas correspondientes al amigo de Pickles que tiene mayor altura en toda la tabla.
Aprendizaje	Aprendí a usar la cláusula ***LIMIT*** para recortar los resultados y obtener solo la cantidad de filas que necesito, sobre todo cuando se combina con ***ORDER BY*** para mostrar al elemento más alto o más bajo de la lista.
Aplicación en el mundo real	Es muy útil en sistemas reales; por ejemplo, en una escuela para consultar los datos del alumno con la calificación más alta. En lugar de revisar toda la tabla con los datos de todos los estudiantes y sus calificaciones, es más fácil filtrarlo con este comando para que muestre de inmediato solo la fila y los datos del alumno de mejor promedio.

Número y nombre del ejercicio	**Ejercicio 12 – COUNT(*)**
Objetivo del ejercicio	Ejecutar una consulta para obtener el número total de filas de la tabla friends_of_pickles, devolviendo una cifra exacta con la cantidad de filas.
Concepto SQL	Practicamos la función **COUNT(*)**, la cual sirve para explorar y verificar cuántas filas tiene una tabla de forma rápida. El uso del asterisco (*) le indica a SQL que cuente absolutamente todos los registros en conjunto para arrojar el total de elementos de la tabla.
Consulta ejecutada	SELECT COUNT(*) FROM friends_of pickles;
Interpretación	El resultado de la consulta muestra de manera exitosa el número total de filas de la tabla, en este caso un total de 7 filas, cumpliendo con el propósito de cuantificar la información.
Aprendizaje	Aprendí a utilizar la función COUNT(*) para contar de manera rápida y automática todas las filas de una tabla, lo que resulta muy práctico para obtener estadísticas rápidas sin tener que revisar los datos uno por uno.
Aplicación en el mundo real	Es muy útil en sistemas reales; por ejemplo, en una plataforma digital para saber exactamente cuántos usuarios están registrados en total.

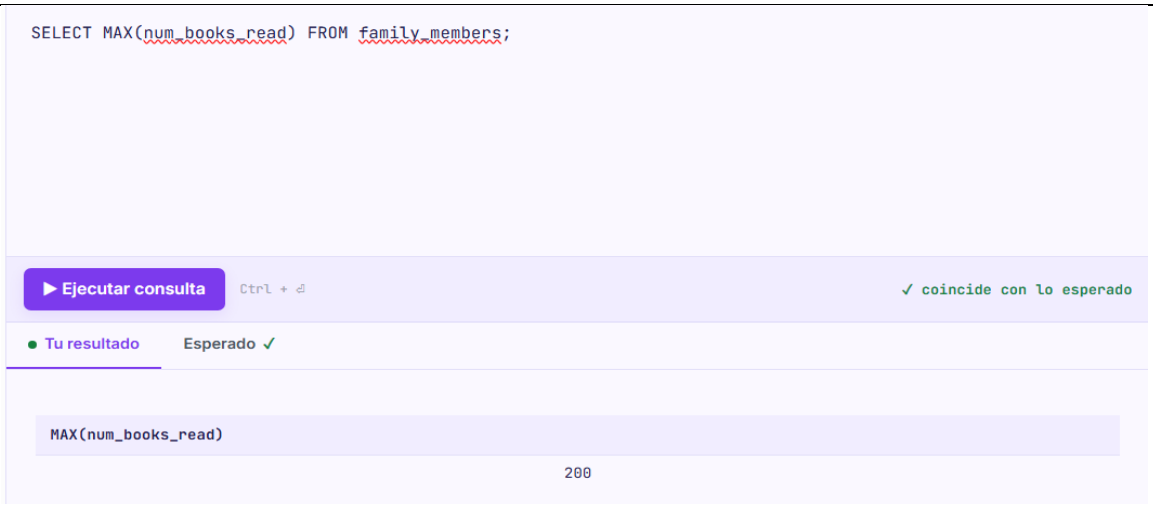
Resultado	

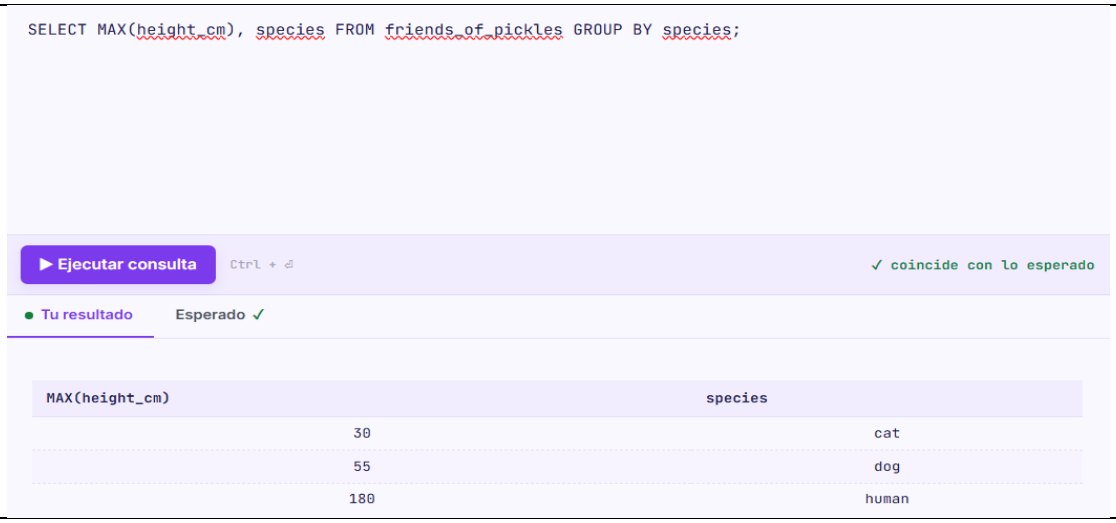
Número y nombre del ejercicio	**Ejercicio 13 – COUNT(*) … WHERE**
Objetivo del ejercicio	Ejecutar una consulta para devolver el número total de filas en la tabla ***friends_of_pickles*** donde la especie sea específicamente un dog, obteniendo la cantidad exacta que cumple con esa condición.
Concepto SQL	Practicamos la combinación de la función ***COUNT(*)*** con la cláusula ***WHERE***, lo cual nos permite contar únicamente las filas que coinciden con un criterio o filtro específico en lugar de sumar todos los registros de la tabla de golpe.
Consulta ejecutada	**SELEC COUNT(*) FROM Friends_of_pickles WHERE species = ‘dog’;**
Resultado	
Interpretación	El resultado de la consulta muestra el número total de registros que cumplen con el filtro indicado, en este caso arroja el número 3, que indica cuántos perros hay registrados en la tabla.
Aprendizaje	Aprendí a usar ***COUNT(*)*** junto con ***WHERE*** para hacer conteos aplicando filtros, lo que me ayuda a obtener la cantidad exacta de datos que necesito sin tener que revisar toda la tabla de forma manual.
Aplicación en el mundo real	Un ejemplo de aplicación seria cuando en una plataforma escolar, usamos estas cláusulas en una consulta, para contar automáticamente cuántos alumnos están inscritos en una materia específica.

Número y nombre del ejercicio	**Ejercicio 14 - SUM**
Objetivo del ejercicio	Ejecutar una consulta para obtener la suma total de los valores de la columna de ***num_books_read*** en la tabla ***family_members***, calculando el total acumulado de toda la familia.
Concepto SQL	Practicamos la función ***SUM***, la cual sirve para sumar de manera automática todos los valores numéricos que se encuentran dentro de una columna específica de la tabla.
Consulta ejecutada	**SELECT SUM(num_books_read) FROM family_members;**
Resultado	
Interpretación	El resultado de la consulta muestra de manera exitosa la suma total de los elementos de esa columna, en este caso fue un total de 380 libros leídos entre los integrantes de la familia.
Aprendizaje	Aprendí a usar la función ***SUM*** para sumar columnas con datos numéricos de forma automática, lo que me ayuda a obtener totales globales sin tener que hacer cuentas manuales registro por registro.
Aplicación en el mundo real	Es muy útil en sistemas reales, por ejemplo: en una tienda en línea para calcular el total de dinero recaudado en las ventas del día, o en una empresa para sumar las horas extras trabajadas por los empleados en un mes.

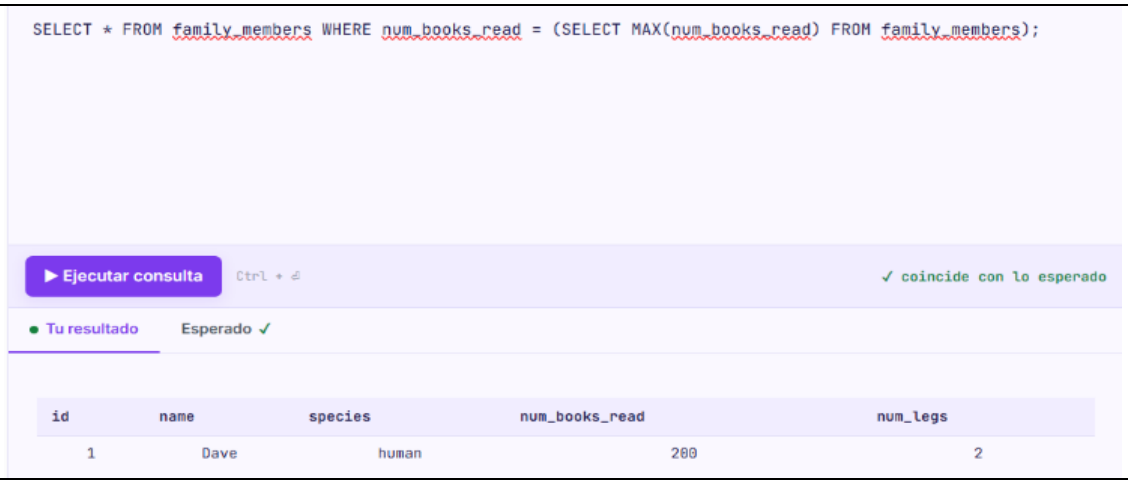
Número y nombre del ejercicio	**Ejercicio 15 - AVG**
Objetivo del ejercicio	Ejecutar una consulta para encontrar el promedio del ***num_books_read*** (número de libros leídos) por cada integrante de la familia en la tabla ***family_members***.
Concepto SQL	En este ejercicio practicamos la función ***AVG***, la cual sirve para calcular de manera automática el promedio de los valores numéricos que se encuentran dentro de una columna específica de la tabla
Consulta ejecutada	**SELECT AVG(num_books_read) FROM family_members;**
Interpretación	El resultado de la consulta nos muestra el promedio calculado de los elementos de esa columna, arroja un resultado decimal de 126.666666666667.
Aprendizaje	Aprendí a usar la función ***AVG*** para obtener promedios de columnas numéricas, lo que me ayuda a conocer el promedio de un valor dado sin tener que hacer operaciones manuales de suma y división registro por registro.
Aplicación en el mundo real	Puede ser muy útil en sistemas reales; por ejemplo, en una escuela para calcular el promedio general de calificaciones de un grupo de estudiantes.
Resultado	

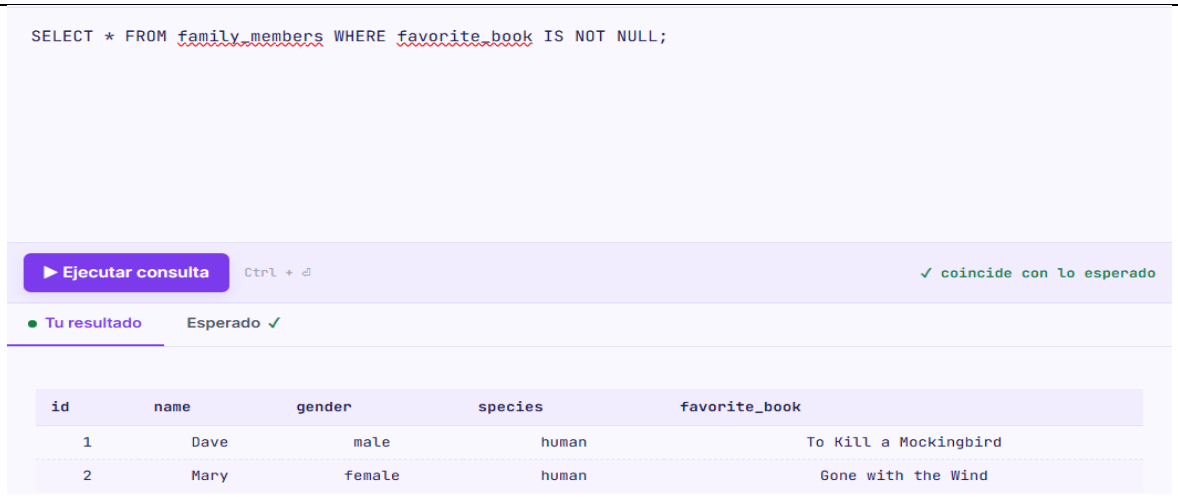
	

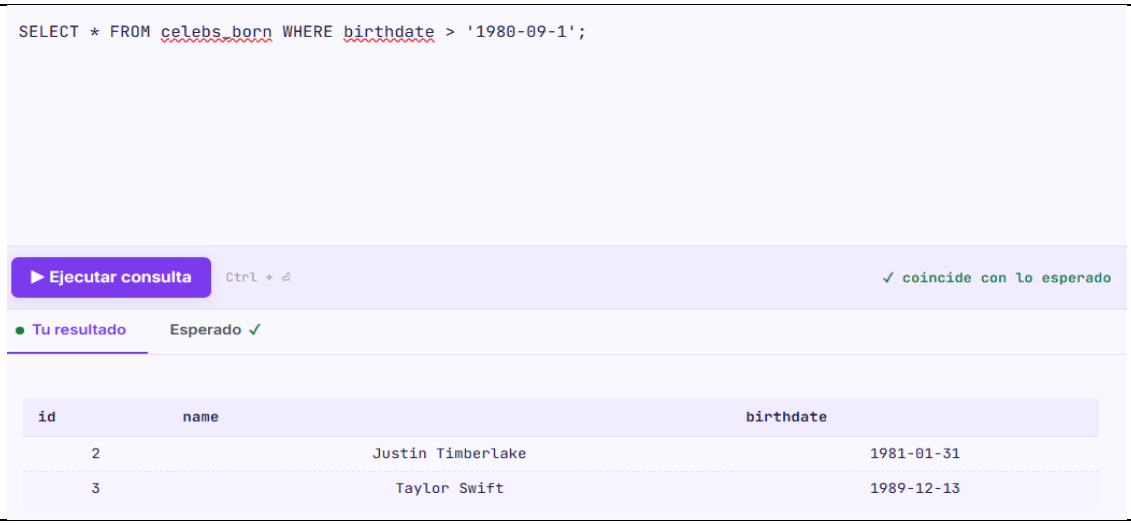
Número y nombre del ejercicio	**Ejercicio 16 – MAX y MIN**
Objetivo del ejercicio	Ejecutar una consulta para encontrar el número de libros leídos más alto que tiene un miembro de la familia en la tabla ***family_members.***
Concepto SQL	En este ejercicio aprendimos a usar las funciones ***MAX y MIN***, las cuales sirven para encontrar de forma automática el valor más alto (máximo) o más bajo (mínimo) que se encuentre dentro de una columna numérica de la tabla.
Consulta ejecutada	**SELECT MAX(num_books_read) FROM family_members;**
Resultado	
Interpretación	El resultado de la consulta muestra de manera exitosa el valor de la cantidad más alta de libros leídos por un integrante que es 200.
Aprendizaje	Aprendí a usar la función ***MAX*** para obtener el valor límite más alto y ***MIN*** para el valor limite más bajo de una columna de forma directa, lo que me ayuda a identificar extremos sin tener que revisar manualmente todos los registros.
Aplicación en el mundo real	Es muy útil en sistemas reales; por ejemplo, en una tienda en línea para consultar rápidamente cuál es el producto más caro del catálogo

Número y nombre del ejercicio	**Ejercicio 17 – GROUP BY**
Objetivo del ejercicio	Ejecutar una consulta para encontrar la altura más alta de cada especie de la tabla ***friends_of_pickles***, devolviendo el nombre de la especie junto con su respectiva altura y agrupadas por categoría.
Concepto SQL	Estamos practicando la cláusula ***GROUP BY***, la cual sirve para dividir los datos de una tabla en diferentes grupos basándose en el valor de una columna específica. Esto nos permite combinarla con funciones de agregación (como ***MAX, COUNT, SUM***, etc.) para obtener cálculos o resúmenes por cada categoría.
Consulta ejecutada	**SELECT MAX(height_cm), species FROM Friends_of_pickles GROUP BY species;**
Resultado	
Interpretación	El resultado de la consulta muestra de manera exitosa los datos divididos por categorías, arrojando la altura máxima correspondiente para cada especie (cat, dog y human) de forma ordenada.
Aprendizaje	Aprendí a utilizar ***GROUP BY*** para agrupar registros y sacar estadísticas por categorías, lo que me ayuda a organizar mejor la información cuando necesito ver resultados separados por grupos en lugar de un total general de toda la tabla.
Aplicación en el mundo real	Un ejemplo sería cuando una tienda en línea la utiliza para calcular el precio del producto más caro de cada departamento por separado.

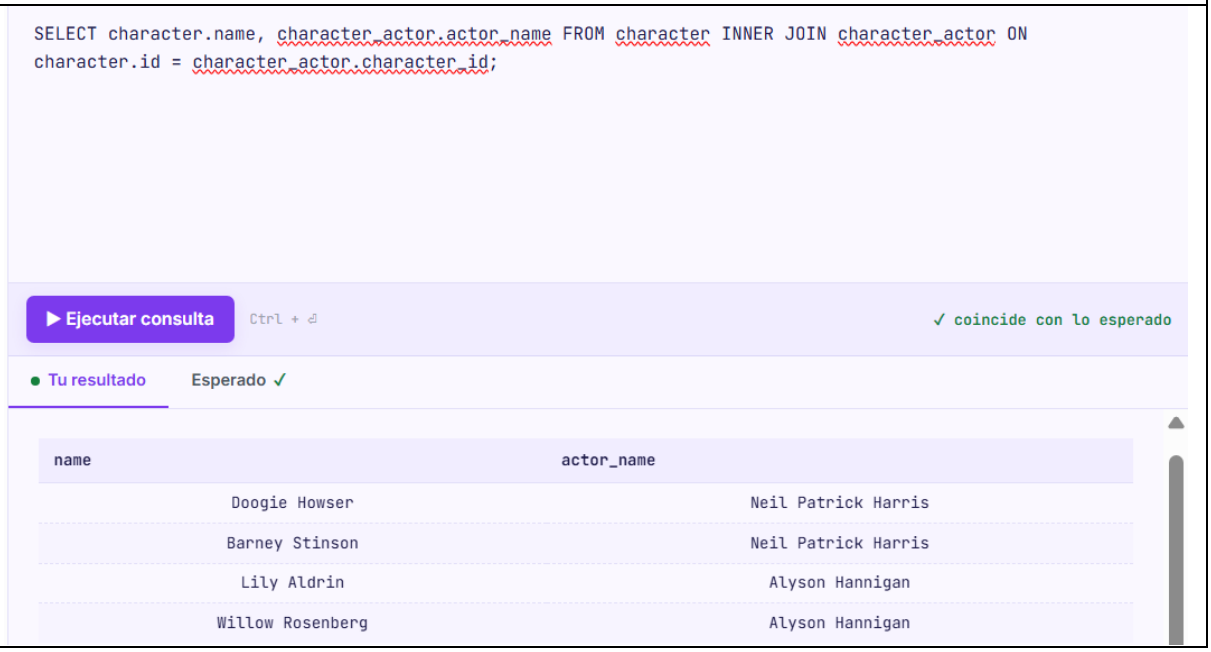
Número y nombre del ejercicio	**Ejercicio 18 -Consultas anidadas**
Objetivo del ejercicio	Ejecutar una consulta para devolver los miembros de la familia que tienen el mayor número de libros leídos, utilizando una consulta dentro de otra para encontrar ese registro máximo.
Concepto SQL	Consiste en aprender y practicar las **consultas anidadas**, las cuales permiten meter una consulta SQL dentro de otra. Primero se ejecuta la consulta que está entre paréntesis para sacar un valor clave (como el número más alto), y ese resultado se usa en la consulta principal para filtrar y traer los datos exactos que buscamos.
Consulta ejecutada	
Interpretación	El resultado de la consulta muestra de manera exitosa la fila completa con todos los datos del integrante de la familia que leyó más libros, que fue Dave con 200 libros.
Aprendizaje	Aprendí a usar consultas anidadas para resolver problemas donde necesito comparar un valor contra un resultado calculado previamente.
Aplicación en el mundo real	En una plataforma de música para consultar el perfil exacto de la persona que acumuló más horas de reproducción.

Resultado	

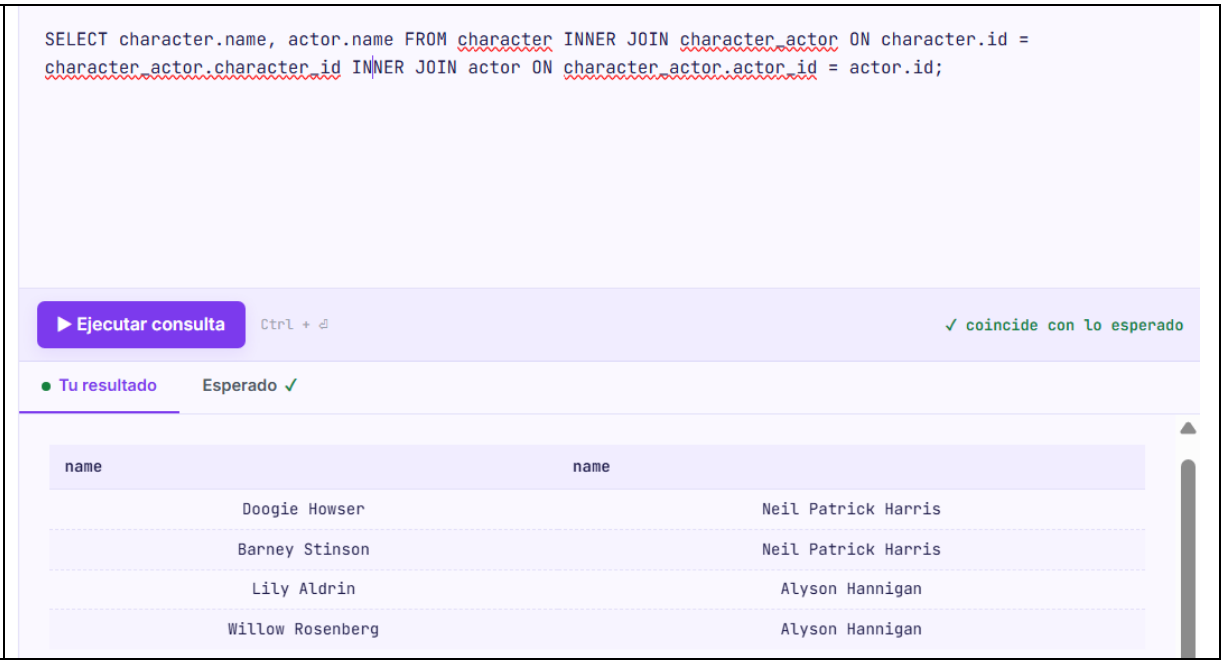
Número y nombre del ejercicio	**Ejercicio 19 - NULL**
Objetivo del ejercicio	Ejecutar una consulta para devolver todas las filas de la tabla ***family_members*** donde el campo de ***favorite_book*** no sea null, es decir, filtrar los registros que sí tengan un valor asignado en esa columna.
Concepto SQL	Practicamos el manejo de valores vacíos mediante las condiciones ***IS NULL*** o ***IS NOT NULL***. Esto sirve para buscar filas donde una columna no tiene ningún valor absoluto (aparece como NULL) o para excluir esos espacios vacíos y traer únicamente los registros que sí contienen información.
Consulta ejecutada	**SELECT * FROM family_members WHERE favorite_book IS NOT NULL;**
Resultado	
Interpretación	El resultado de la consulta muestra de manera exitosa las filas filtradas de la tabla, omitiendo los registros vacíos y mostrando únicamente a los miembros que sí tienen un libro favorito registrado.
Aprendizaje	Aprendí a usar la condición ***IS NOT NULL*** para limpiar mis búsquedas y evitar que aparezcan datos vacíos o incompletos, lo que me ayuda a trabajar únicamente con los registros que sí tienen información útil.
Aplicación en el mundo real	Es muy útil en varias situaciones; por ejemplo, en una base de datos de una escuela para filtrar y mostrar únicamente a los alumnos que sí entregaron un proyecto o tarea, dejando fuera los registros de quienes no enviaron nada.

Número y nombre del ejercicio	**Ejercicio 20 - Fecha**
Objetivo del ejercicio	Ejecutar una consulta para devolver la lista de celebridades que nacieron después del 1 de septiembre de 1980 en la tabla ***celebs_born.***
Concepto SQL	Estamos practicando el filtrado con valores de fecha usando operadores de comparación como **< >.** Esto nos permite comparar datos temporales (en formato AAAA-MM-DD) para traer únicamente los registros que ocurrieron antes o después de un día específico.
Consulta ejecutada	**SELECT * FROM celebs_born WHERE birthdate > '1980-09-1';**
Resultado	
Interpretación	El resultado de la consulta muestra de manera exitosa a las celebridades que nacieron después del 1 de septiembre de 1980 y son: Justin Timberlake y Taylor Swift.
Aprendizaje	Aprendí a comparar fechas usando operadores lógicos como los signos **<>,** lo que me permite hacer filtros cronológicos de forma directa sin tener que procesar el año o el mes por separado.
Aplicación en el mundo real	Es muy útil en situaciones del día a día, por ejemplo: en una plataforma digital para filtrar y mostrar únicamente a los usuarios que se registraron durante el último mes, o en una tienda para consultar las facturas emitidas después de cierta fecha.

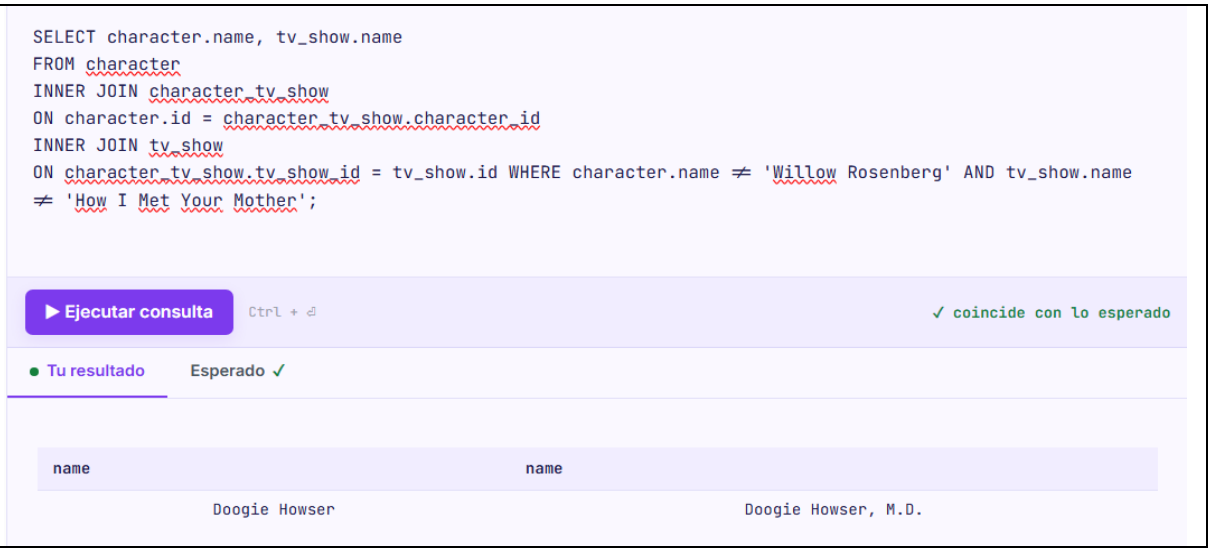
Número y nombre del ejercicio	**Ejercicio 21 – Inner Joins**
Objetivo del ejercicio	Ejecutar una consulta utilizando ***INNER JOIN*** para emparejar y mostrar el nombre de cada personaje junto con el nombre del actor que lo interpreta (character.name y character_actor.actor_name), conectando dos tablas diferentes a través de sus columnas en común.
Concepto SQL	Estamos practicando la instrucción ***INNER JOIN ... ON***, la cual sirve para unir dos tablas distintas relacionándolas por medio de un identificador o columna compartida. Esto nos permite cruzar información que está separada para verla junta en una sola tabla de resultados sin duplicar datos.
Consulta ejecutada	**SELECT character.name, character_actor.actor_name** **FROM character** **INNER JOIN character_actor** **ON character.id = character_actor.character_id;**
Interpretación	El resultado de la consulta muestra de manera exitosa la tabla combinada, relacionando correctamente a cada personaje con su respectivo actor, por ejemplo, Doogie Howser con Neil Patrick Harris, o Lily Aldrin con Alyson Hannigan.
Aprendizaje	Aprendí a usar ***INNER JOIN*** para conectar información de dos tablas diferentes usando una columna en común.

Aplicación en el mundo real	Es muy útil en sistemas reales, ejemplo: en una base de datos de una escuela para unir la tabla de alumnos con la tabla de calificaciones y ver el nombre del estudiante junto con sus materias y notas.
Resultado	

Número y nombre del ejercicio	**Ejercicio 22 – Múltiples joins**
Objetivo del ejercicio	Ejecutar una consulta utilizando ***INNER JOIN,*** para emparejar el nombre de cada personaje con el actor que lo interpreta (***character.name*** y ***actor.name),*** conectando varias tablas a través de sus relaciones intermedias.
Concepto SQL	Practicamos el uso de **múltiples INNER JOIN** en una sola consulta. Esto sirve cuando la información está separada en varias tablas y necesitamos unirlas en cadena mediante tablas de relación para cruzar datos complejos sin duplicar información.
Consulta ejecutada	**SELECT character.name, actor.name FROM character INNER JOIN character_actor ON character.id = character_actor.character_id INNER JOIN actor ON character_actor.actor_id = actor.id;**
Interpretación	El resultado de la consulta muestra de manera exitosa la tabla combinada con los datos de las diferentes fuentes, uniendo de forma correcta a cada personaje con su respectivo actor de la vida real.
Aprendizaje	Aprendí a encadenar varios ***INNER JOIN*** para relacionar más de dos tablas al mismo tiempo, lo que me permite armar consultas más completas cuando la información está distribuida en diferentes partes de una base de datos
Aplicación en el mundo real	En una plataforma de comercio electrónico para generar un reporte completo que una en una sola tabla los datos del cliente, los productos que compró y la información de la empresa de envíos encargada de entregarlos.

Resultado	

Número y nombre del ejercicio	**Ejercicio 23 – Joins con WHERE**
Objetivo del ejercicio	Ejecutar una consulta combinando ***INNER JOIN*** con la cláusula ***WHERE*** para obtener una lista de personajes y programas de TV filtrando los resultados, excluyendo específicamente a los que se llamen "Willow Rosenberg" o estén en el programa "How I Met Your Mother".
Concepto SQL	Nos enseña a utilizar la combinación de ***JOIN*** con la cláusula ***WHERE***, lo cual nos permite unir varias tablas para cruzar su información y al mismo tiempo, aplicar filtros de exclusión o condiciones específicas para quedarnos únicamente con los datos que nos interesan.
Consulta ejecutada	**SELECT character.name, tv_show.name** **FROM character** **INNER JOIN character_tv_show** **ON character.id = character_tv_show.character_id** **INNER JOIN tv_show** **ON character_tv_show.tv_show_id = tv_show.id WHERE character.name != 'Willow Rosenberg' AND tv_show.name != 'How I Met Your Mother';**
Interpretación	El resultado de la consulta muestra de manera exitosa la relación entre los personajes y sus programas de TV ya depurada, omitiendo los registros que cumplían con las condiciones de exclusión aplicadas en el filtro.
Aprendizaje	Aprendí a mezclar uniones de tablas con filtros de selección, lo que me ayuda a limpiar los resultados cruzados cuando no quiero que aparezcan ciertos datos específicos en la tabla final.
Aplicación en el mundo real	Es muy útil en sistemas reales, como por ejemplo, en una tienda en línea para generar un reporte que una las ventas con los datos de los clientes, pero excluyendo a los compradores de una ciudad en específico.

Resultado	

Número y nombre del ejercicio	**Ejercicio 24 – Left joins**
Objetivo del ejercicio	Ejecutar una consulta utilizando ***LEFT JOIN*** para emparejar el nombre de cada personaje con el actor que lo interpreta, asegurando que se muestren todos los registros de la tabla principal aunque no tengan una coincidencia exacta.
Concepto SQL	Practicamos la instrucción ***LEFT JOIN,*** la cual sirve para unir tablas asegurando que se devuelvan absolutamente todos los datos de la primera tabla (lado izquierdo). Si no hay un dato correspondiente en la tabla secundaria, la consulta de todas formas muestra el registro principal y coloca un valor **NULL** en los espacios vacíos.
Consulta ejecutada	**SELECT character.name, actor.name** **FROM character** **LEFT JOIN character_actor** **ON character.id = character_actor.character_id** **LEFT JOIN actor** **ON character_actor.actor_id = actor.id;**
Interpretación	El resultado de la consulta muestra de manera exitosa la lista completa de personajes, relacionando a los actores que tienen asignados y mostrando espacios vacíos (**NULL**) para aquellos personajes que no cuentan con un actor registrado.
Aprendizaje	Aprendí a usar ***LEFT JOIN*** para hacer cruces de tablas sin perder los registros principales, lo que me ayuda a visualizar toda la información base, aunque algunas relaciones estén incompletas o vacías.
Aplicación en el mundo real	Ejemplo: en una escuela para listar a todos los alumnos y mostrar al lado su calificación de un examen, asegurando que aparezcan todos los estudiantes aunque algunos todavía no tengan una nota registrada.

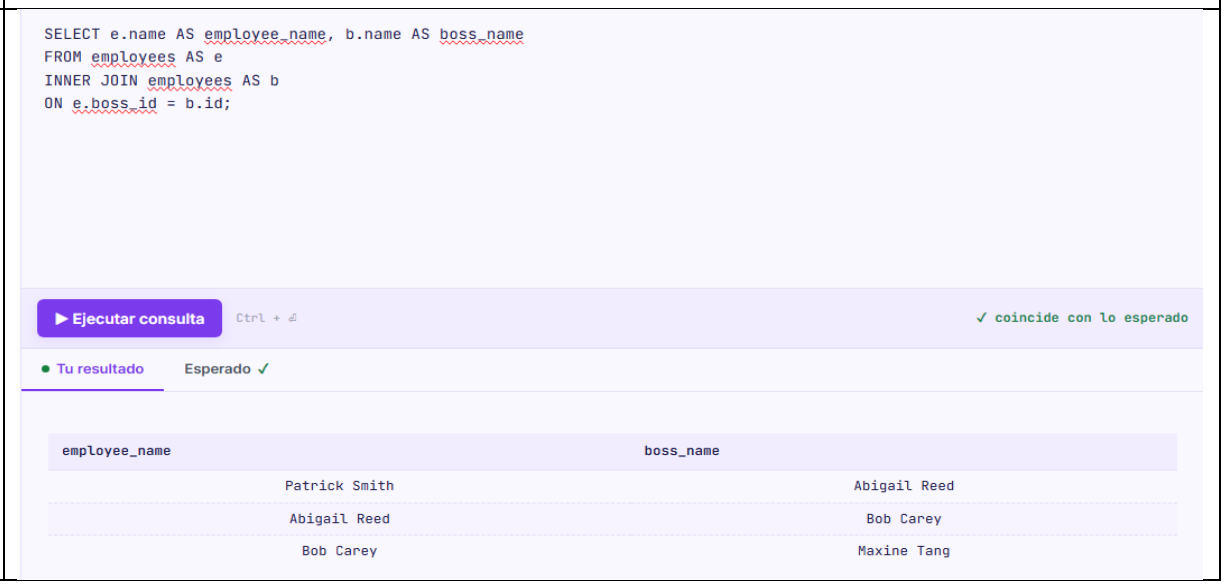
Resultado	

Número y nombre del ejercicio	**Ejercicio 25 – Alias de tabla**
Objetivo del ejercicio	Ejecutar una consulta usando alias (AS) para abreviar los nombres de las tablas, character como c, character_actor como ca, y actor como a, y emparejar los nombres de los personajes con los actores que los interpretan mediante LEFT JOIN, logrando una estructura de código más corta y limpia.
Concepto SQL	Practicamos el uso de **alias de tabla** mediante la palabra clave ***AS***. Esto sirve para crear un "apodo" corto para cada tabla al momento de hacer consultas largas o con varios **JOIN,** lo que evita tener que escribir el nombre completo de la tabla cada vez que necesitamos referenciar una columna.
Consulta ejecutada	**SELECT c.name, a.name** **FROM character AS c** **LEFT JOIN character_actor AS ca** **ON c.id = ca.character_id** **LEFT JOIN actor AS a** **ON ca.actor_id = a.id;**
Interpretación	El resultado de la consulta muestra la misma relación completa entre personajes y actores, pero utilizando una estructura mucho más abreviada, ordenada y fácil de leer.
Aprendizaje	Aprendí a usar alias para renombrar tablas temporalmente dentro de una consulta.
Aplicación en el mundo real	Es muy útil en situaciones del día a día, ejemplo: en una libreta de contactos o directorio telefónico donde guardas los datos de tus amigos y sus apodos cortos, para que al momento de buscar un número o mandar un mensaje rápido no tengas que escribir nombres largos y la información sea más fácil de consultar.

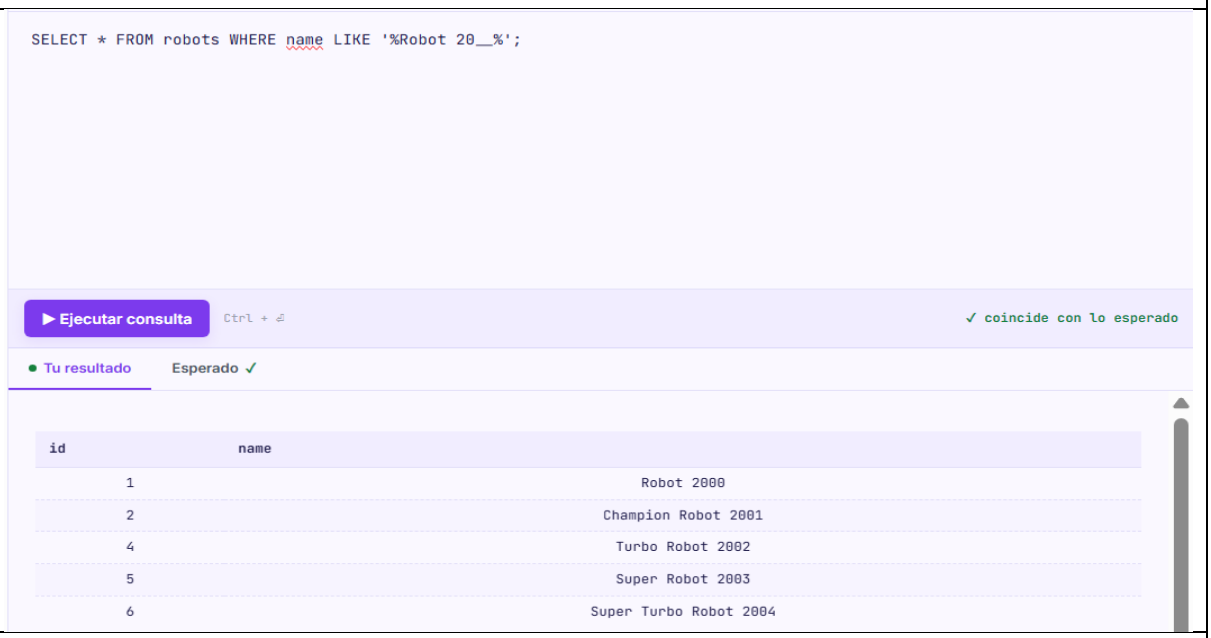
Resultado	

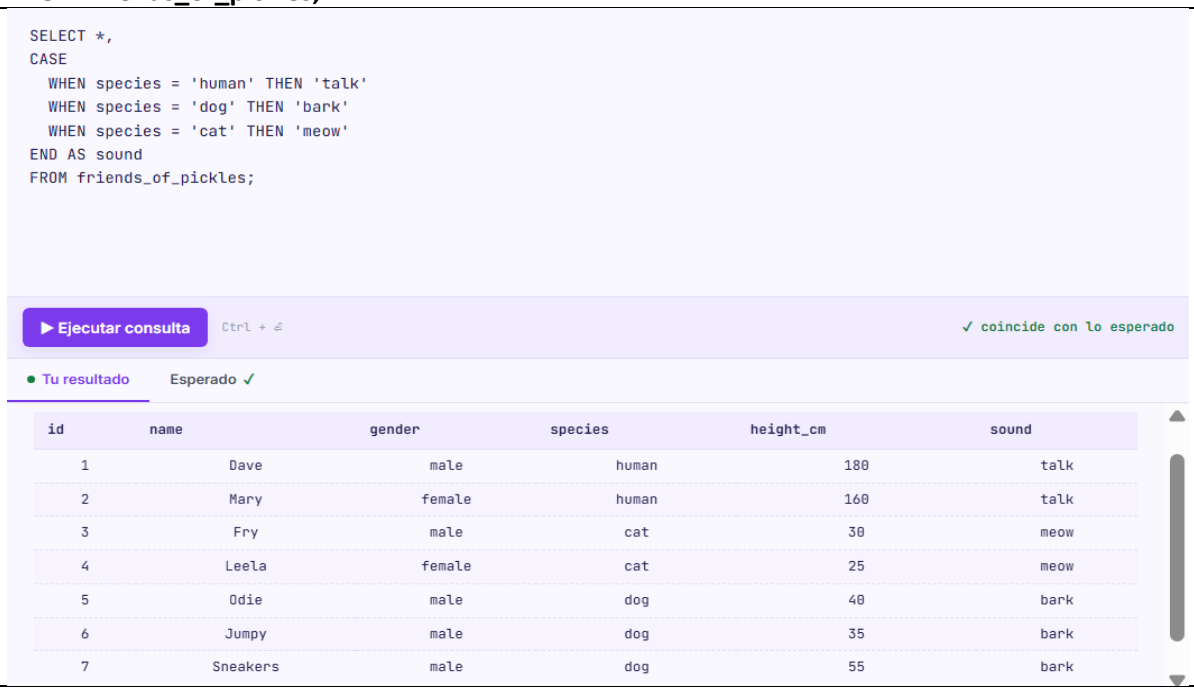
Número y nombre del ejercicio	**Ejercicio 26 - Alias de columna**
Objetivo del ejercicio	Ejecutar una consulta combinando tablas mediante ***LEFT JOIN*** y asignando alias a las columnas (**character.name** AS **character** y **actor.name** AS **actor**) para renombrar los encabezados del resultado y evitar confusiones al mostrar la información.
Concepto SQL	Nos enseña el uso de **alias de columna** mediante la instrucción ***AS***. Esto sirve para cambiar temporalmente el nombre que aparece en el encabezado de los resultados, lo cual es ideal cuando seleccionamos columnas de distintas tablas que se llaman igual (como name) para identificarlas fácilmente.
Consulta ejecutada	**SELECT character.name AS character, actor.name AS actor** **FROM character** **LEFT JOIN character_actor** **ON character.id = character_actor.character_id** **LEFT JOIN actor** **ON character_actor.actor_id = actor.id;**
Interpretación	El resultado de la consulta muestra de manera exitosa la lista de personajes y sus respectivos actores, pero ahora con los encabezados renombra dos claramente como character y actor, lo que hace que la tabla final sea mucho más entendible.
Aprendizaje	Aprendí a renombrar las columnas en los resultados finales usando alias, lo que me ayuda a mantener la información organizada y clara cuando hago consultas que unen varias tablas con nombres de campos repetidos.
Aplicación en el mundo real	Es muy útil en situaciones del día a día; por ejemplo, al generar una lista de contactos que incluya el teléfono del alumno y el del tutor, usando alias para que los encabezados digan "Teléfono Alumno" y "Teléfono Tutor" en lugar de mostrar dos columnas llamadas simplemente "Teléfono".

Resultado	

Número y nombre del ejercicio	**Ejercicio 27 – Self joins**
Objetivo del ejercicio	Ejecutar una consulta que devuelva el nombre de un empleado y el nombre de su jefe, usando alias de columna para hacer las columnas ***employee_name*** y ***boss_name*****.**
Concepto SQL	Practicamos el uso de **Self join** (unir una tabla consigo misma). Esto sirve cuando una sola tabla contiene datos que se relacionan entre sí, por lo que asignamos un alias diferente a cada copia de la misma tabla (como e para empleados y b para jefes) para poder cruzar los registros de la misma lista sin causar confusiones.
Consulta ejecutada	**SELECT e.name AS employee_name, b.name AS boss_name FROM employees AS e INNER JOIN employees AS b ON e.boss_id = b.id;**
Resultado	

Interpretación	El resultado de la consulta muestra de manera exitosa a cada empleado emparejado con el nombre de su jefe correspondiente, como: Patrick Smith con Abigail Reed, presentando los encabezados claros y organizados.
Aprendizaje	Aprendí a hacer uniones sobre una misma tabla usando alias diferentes para cada instancia, lo que me permite resolver relaciones jerárquicas o dependencias dentro de un mismo grupo de datos de forma directa.
Aplicación en el mundo real	Es muy útil en situaciones del día a día; por ejemplo, en un torneo escolar o deportivo para cruzar la lista de participantes consigo misma y generar los enfrentamientos de cada ronda, mostrando en una columna al jugador y en la otra a su rival asignado.

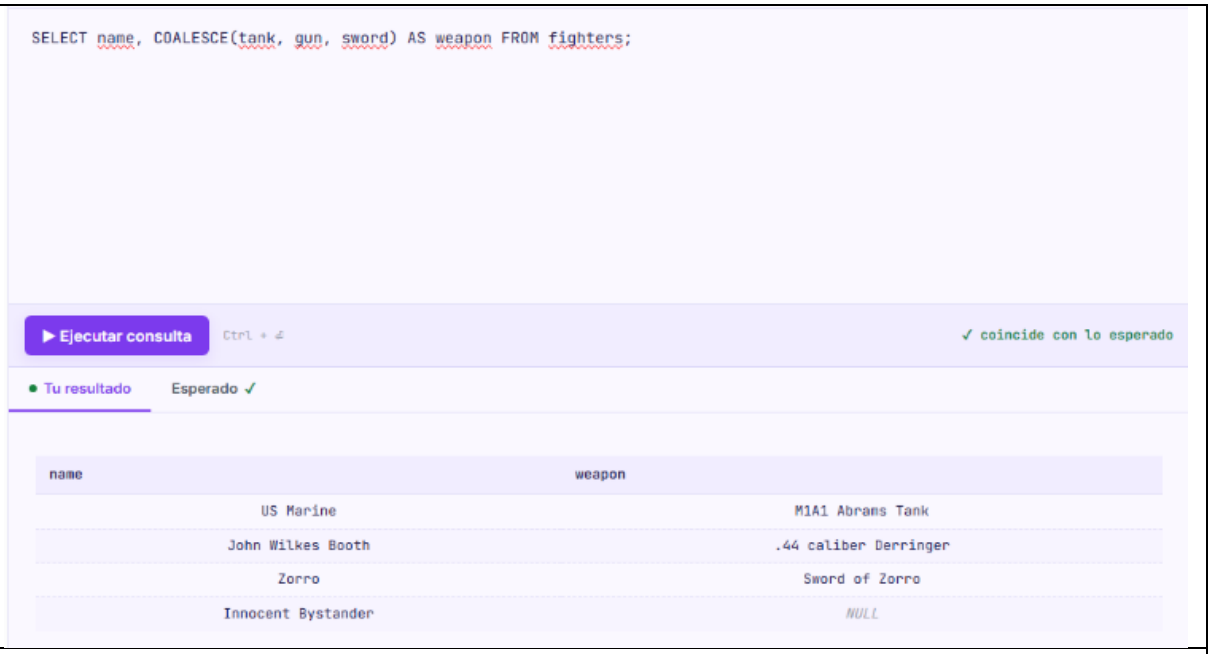
Número y nombre del ejercicio	**Ejercicio 28 - LIKE**
Objetivo del ejercicio	Ejecutar una consulta en la tabla robots utilizando la cláusula LIKE junto con los operadores de comparación % y _ para buscar y devolver los registros cuyo nombre contenga la palabra "Robot" seguida de un año específico entre 2000 y 2099.
Concepto SQL	Practicamos el uso del comando **LIKE** con patrones de texto. Esto sirve para hacer búsquedas flexibles usando caracteres especiales como el porcentaje (%) para representar múltiples caracteres y el guion bajo (_) para representar un carácter exacto, lo que nos permite encontrar coincidencias parciales sin necesidad de saber el texto exacto.
Consulta ejecutada	**SELECT * FROM robots WHERE name LIKE '%Robot 20__%';**
Resultado	
Interpretación	El resultado de la consulta muestra de manera exitosa los robots cuyos nombres cumplen con el patrón de búsqueda establecido, filtrando correctamente los registros que coinciden con la estructura de texto indicada.
Aprendizaje	Aprendí a utilizar el operador LIKE y sus operadores de comparación para hacer búsquedas basadas en patrones de texto, lo que me da la libertad de encontrar información cuando solo recuerdo una parte del dato o necesito buscar por coincidencias parciales.
Aplicación en el mundo real	En un buscador de contactos o directorio para escribir las primeras letras de un nombre y encontrar automáticamente a todas las personas que coinciden con esa búsqueda sin tener que teclear el nombre completo.

Número y nombre del ejercicio	**Ejercicio 29 - CASE**
Objetivo del ejercicio	Ejecutar una consulta en la tabla ***friends_of_pickles*** utilizando ***CASE*** para evaluar diferentes condiciones según la especie y generar una nueva columna llamada ***sound*** que devuelva un valor específico ("talk" para humanos, "bark" para dogs y "meow" para cats).
Concepto SQL	Practicamos el uso de la estructura condicional ***CASE***. Esto sirve para crear lógica condicional dentro de una consulta, como un: -"si pasa esto, devuelve aquello", evaluando distintos escenarios y poniéndole valores personalizados a una nueva columna mediante las palabras clave ***WHEN, THEN*** y ***ELSE***, cerrando siempre con ***END***.
Consulta ejecutada	**SELECT *,** **CASE** **WHEN species = 'human' THEN 'talk'** **WHEN species = 'dog' THEN 'bark'** **WHEN species = 'cat' THEN 'meow'** **END AS sound** **FROM friends_of_pickles;**
Resultado	 id name gender species height_cm sound 1 Dave male human 180 talk 2 Mary female human 160 talk 3 Fry male cat 30 meow 4 Leela female cat 25 meow 5 Odie male dog 40 bark 6 Jumpy male dog 35 bark 7 Sneakers male dog 55 bark
Interpretación	El resultado de la consulta muestra de manera exitosa la tabla original acompañada de la nueva columna ***sound,*** la cual clasifica y asigna correctamente el sonido correspondiente a cada fila basándose en la especie de cada personaje.
Aprendizaje	Aprendí a usar la declaración ***CASE*** para transformar datos al vuelo y crear categorías personalizadas según las condiciones que yo elija, lo que me permite enriquecer los resultados de una consulta sin modificar la base de datos original.
Aplicación en el mundo real	Es muy útil en diversas situaciones por ejemplo una seria: en un sistema de calificaciones de una escuela para evaluar la nota numérica de un alumno y mostrar automáticamente un texto como "Aprobado" si sacó 6 o más, o "Reprobado" si sacó menos, sin tener que cambiar la calificación original.

Número y nombre del ejercicio	**Ejercicio 30 - SUBSTR**
Objetivo del ejercicio	Ejecutar una consulta en la tabla robots usando la función ***SUBSTR*** para extraer una porción específica del texto de la columna location (en este caso, los últimos caracteres) y filtrar todos los registros que estén ubicados en "NY".

Concepto SQL	Practicamos el uso de la función ***SUBSTR*** (subcadena). Sirve para recortar y extraer una parte específica de un texto dentro de una columna, indicando desde qué posición queremos empezar a contar, incluso usando números negativos para contar desde el final, y cuántos caracteres queremos obtener.
Consulta ejecutada	**SELECT * FROM robots WHERE SUBSTR(location, -2) = 'NY';**
Resultado	
Interpretación	El resultado de la consulta muestra de manera exitosa a los robots que cumplen con la condición de estar ubicados en Nueva York (NY), filtrando correctamente los datos.
Aprendizaje	Aprendí a usar la función ***SUBSTR*** para aislar y analizar fragmentos específicos de un texto, lo que me permite trabajar con datos que vienen combinados o en formatos largos sin necesidad de modificar la información original de la tabla.
Aplicación en el mundo real	Es muy útil en situaciones del día a día; por ejemplo, al consultar una lista de números telefónicos con clave lada, usando SUBSTR para tomar solo los primeros 3 dígitos del número y saber de qué ciudad o región es cada contacto sin tener que revisar la dirección completa.

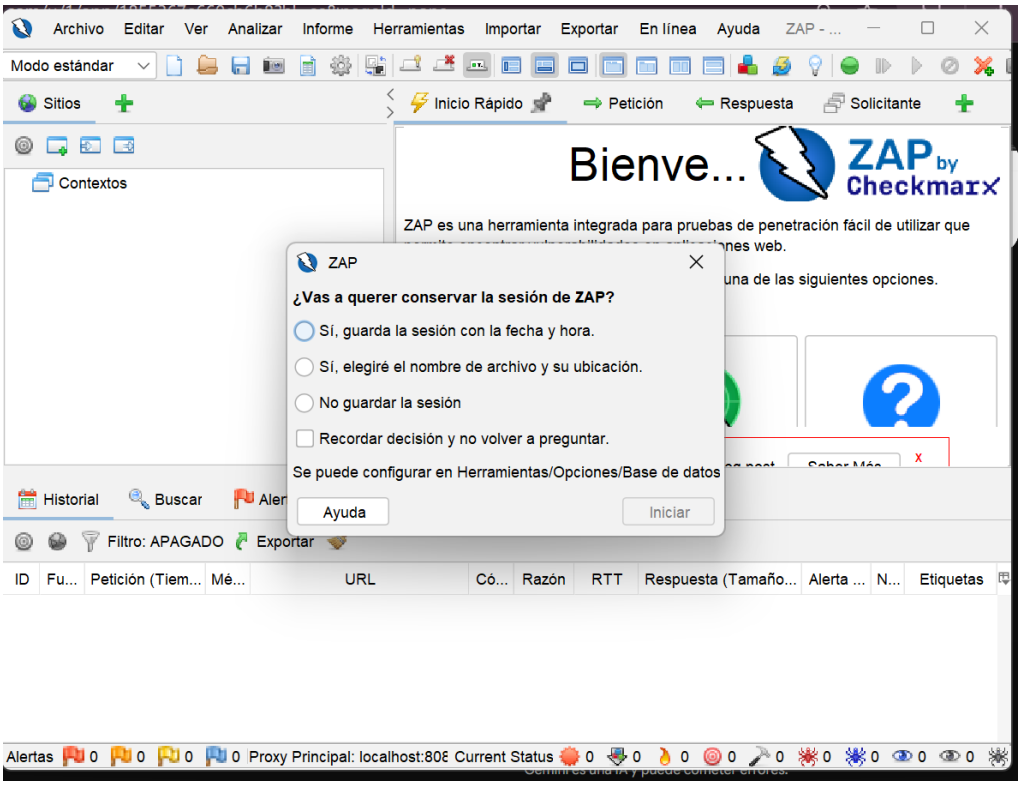
Número y nombre del ejercicio	**Ejrecicio 31 - COALESCE**
Objetivo del ejercicio	Ejecutar una consulta en la tabla ***fighters*** utilizando la función ***COALESCE*** para evaluar las columnas ***tank, gun*** y ***sword*** en orden de prioridad, devolviendo la primera opción que no sea un valor nulo y renombrando el resultado como ***weapon.***
Concepto SQL	Estamos practicando la función ***COALESCE***. Esta función sirve para revisar una lista de columnas o valores en orden y devolver el primer dato que encuentre que no sea **NULL**.
Consulta ejecutada	**SELECT name, COALESCE(tank, gun, sword) AS weapon FROM fighters;**
Interpretación	El resultado de la consulta muestra de manera exitosa a cada luchador con el arma de mayor prioridad que tiene disponible, tomando el tanque si cuenta con uno, o en su defecto el arma de fuego, la espada, o dejando un valor nulo si no tiene ninguna disponible.

Resultado	
Aprendizaje	Aprendí a usar ***COALESCE*** para manejar múltiples opciones de respuesta en una consulta, lo que me ayuda a rescatar información de respaldo sin llenar los resultados de espacios vacíos o depender de condiciones muy complejas.
Aplicación en el mundo real	Es muy útil en situaciones del día a día, ejemplo: en una lista de contactos para buscar primero el teléfono celular de una persona, y si no tiene uno registrado, tomar el teléfono de su casa o el de su trabajo como alternativa en la misma columna.

DOCUMENTACION OWASP ZAP

1- PREPARA TU ENTORNO

1.1 - ZAP abierto

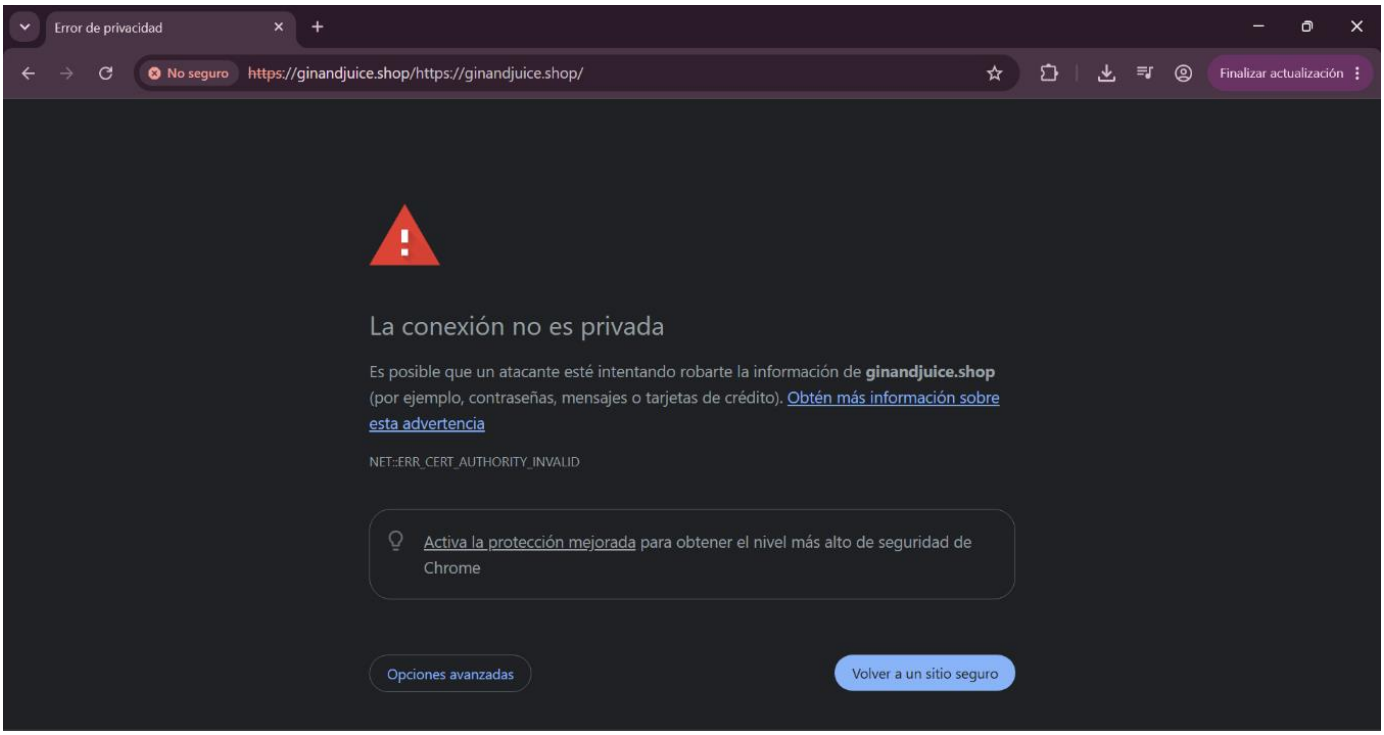


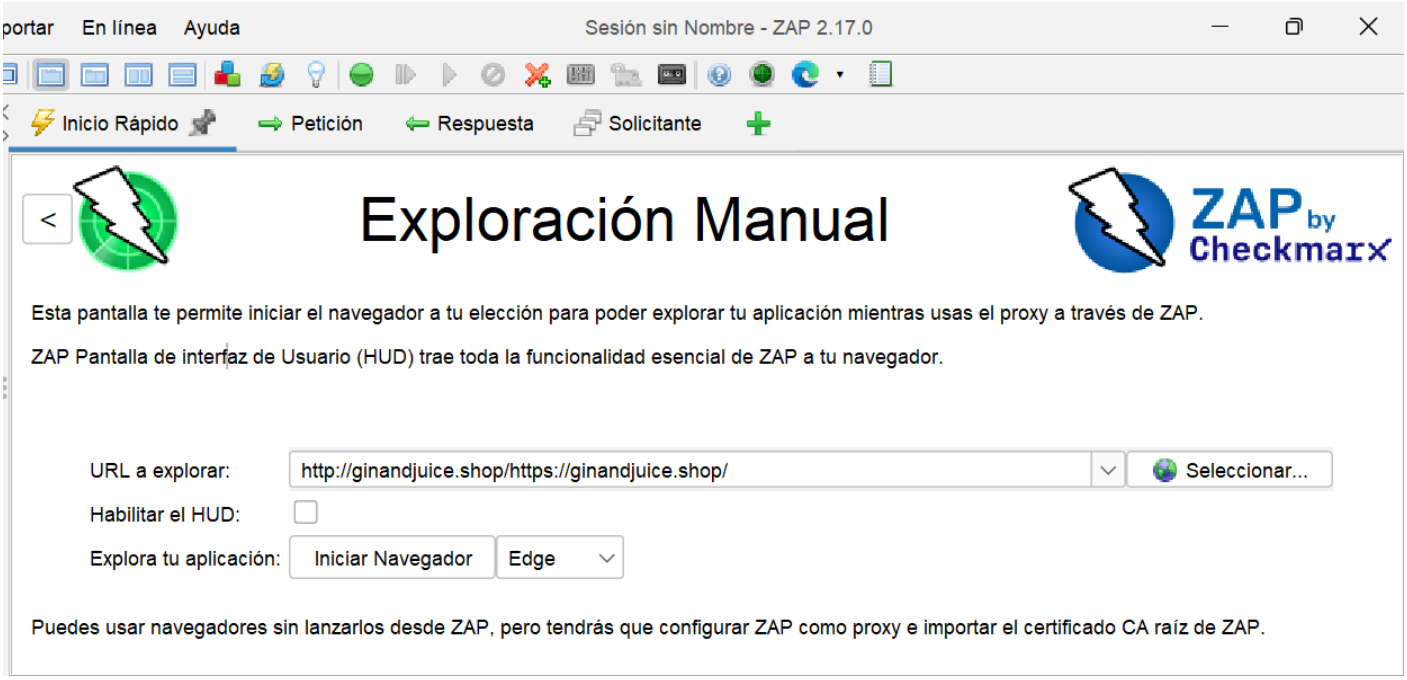
1.2- Navegador configurado

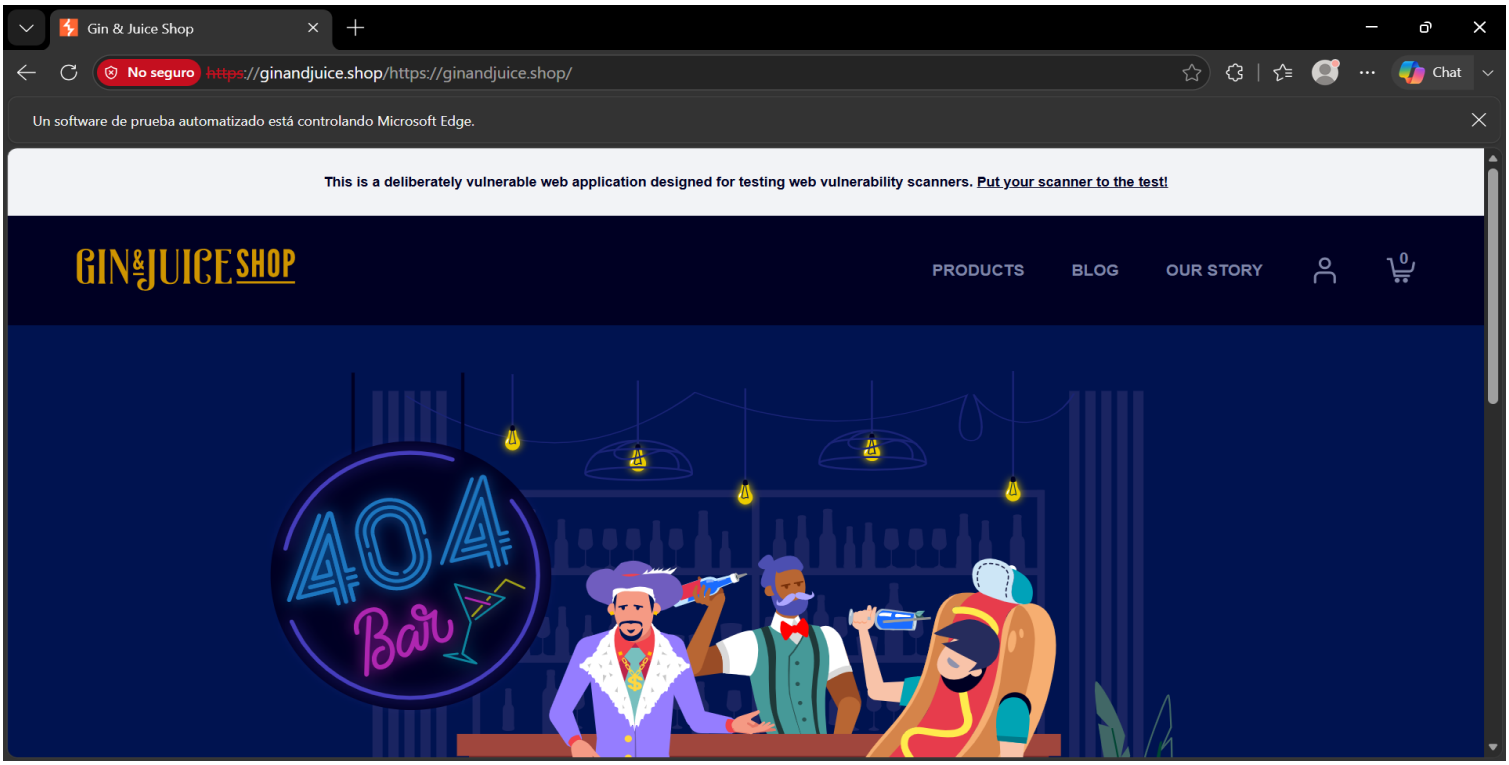


1.3- Acceso a https://ginandjuice.shop/https://ginandjuice.shop/

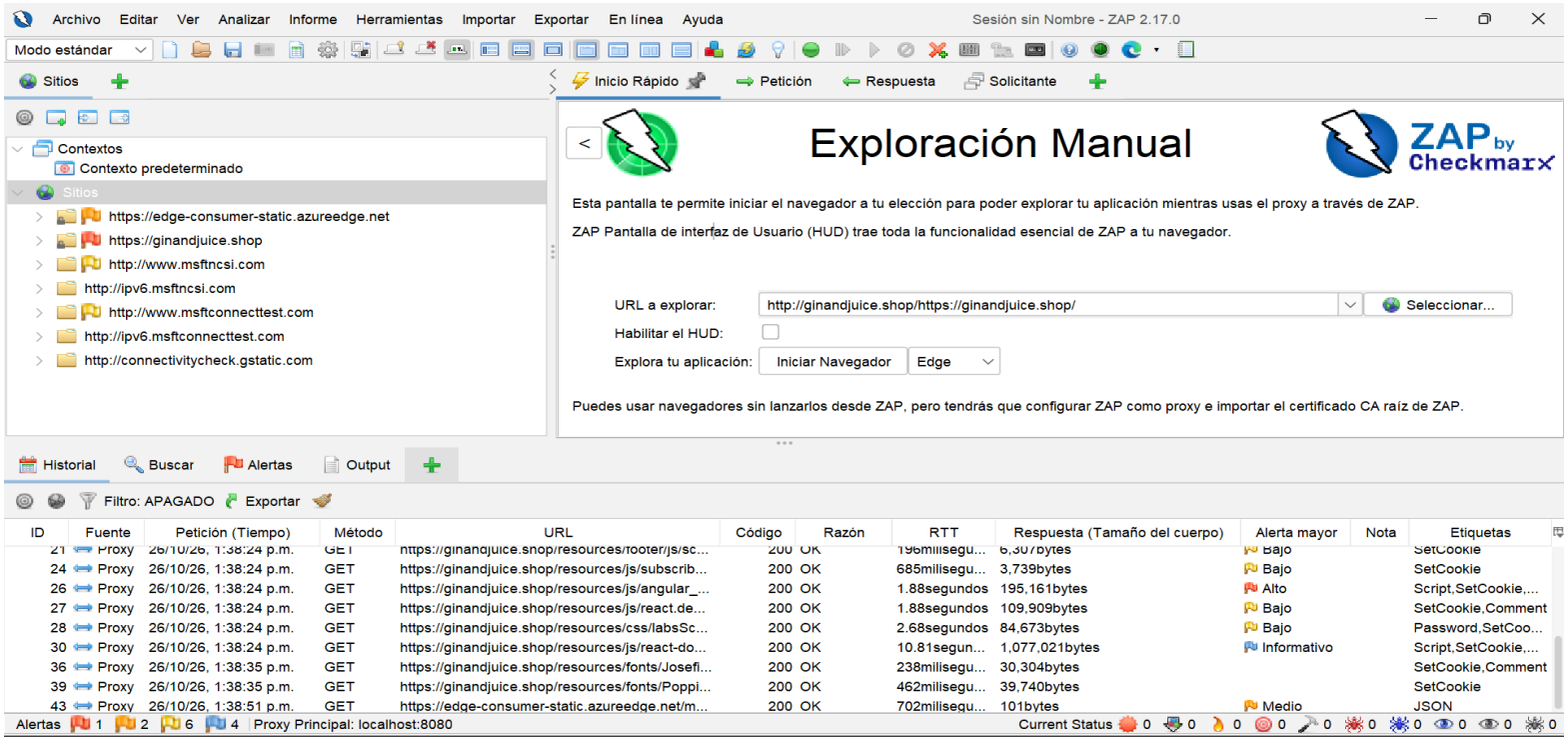
En este paso tuve problemas, al iniciar la practica e intentar cargar el sitio la pantalla se bloqueo mostrando un error de “La conexión no es privada”, lo que impedía el acceso a la plataforma. Para resolver esto, configure el proxy local en OWASP ZAP para canalizar el tráfico de navegación, por lo que se utilizo le herramienta de navegador preconfigurado integrada en OWASP ZAP, al abrir la página desde ese entorno logre acceder exitosamente a la pagina de la tienda Gin & Juice Shop para registrar el trafico dentro del proxy, manteniéndose el candado rojo en la barra de navegación como advertencia de que seguía sin ser segura al estar capturando datos desde OWASP ZAP.







1.4- Verificación de que el trafico aparece en ZAP

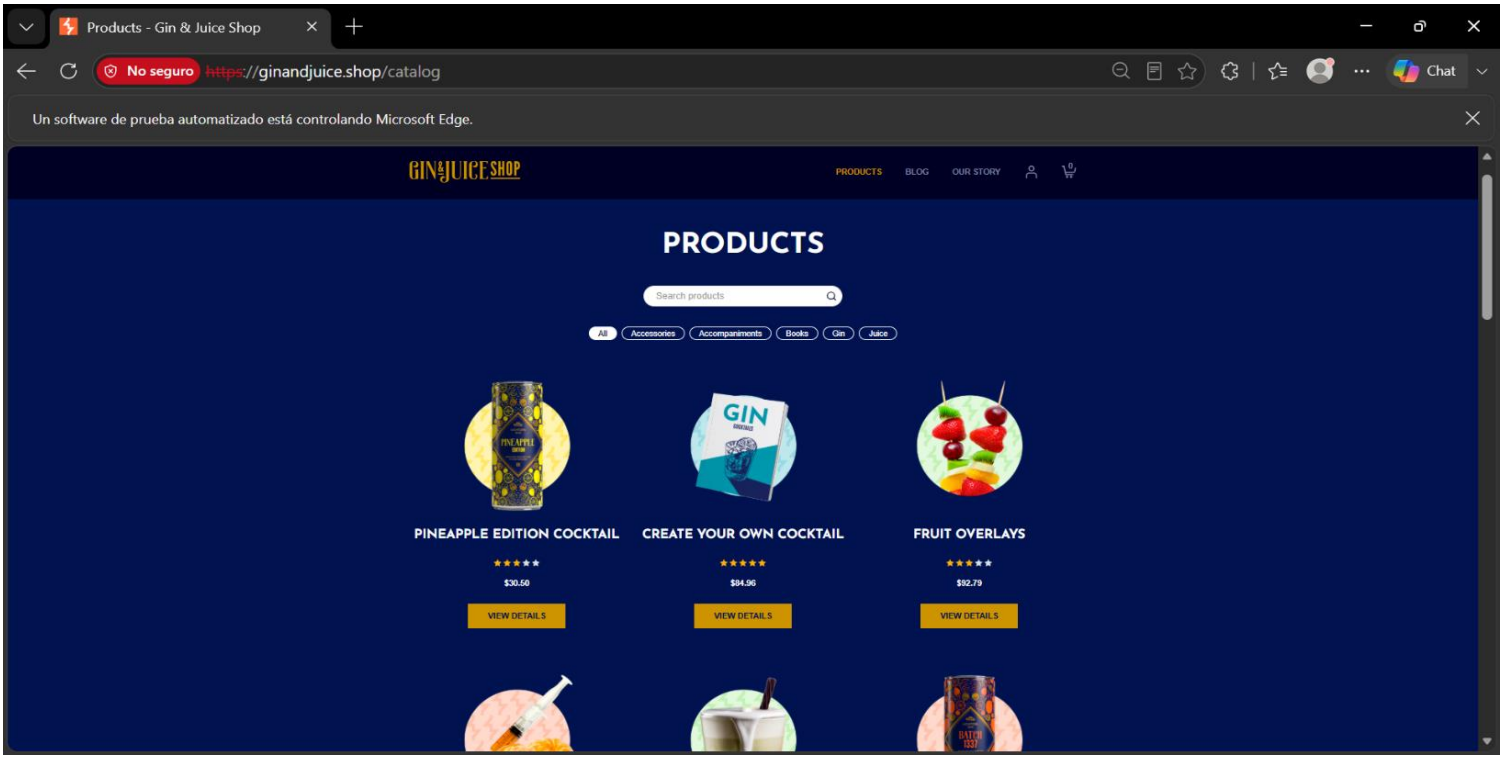


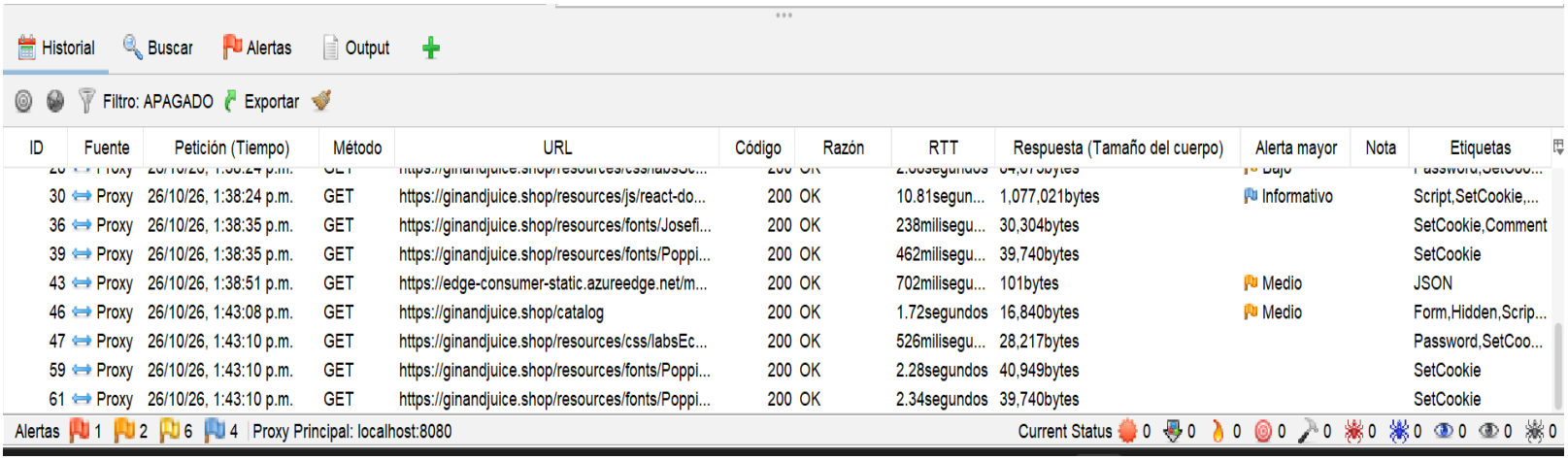
2- EXPLORA LA APLICACIÓN

2.1- Navegación en el sitio y Trafico HTTP en ZAP

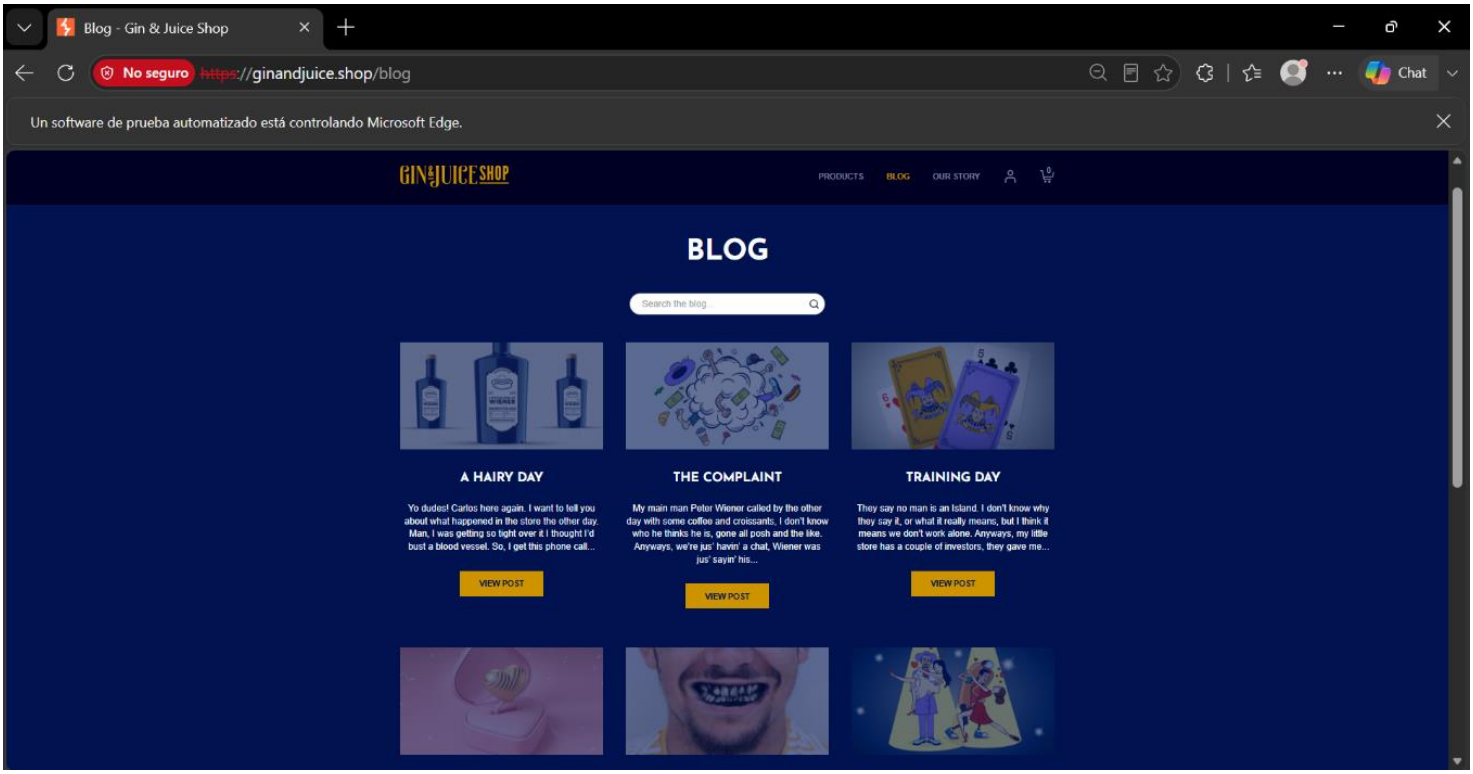
Realice una navegación por toda la página revisando todas y cada una de las secciones con las que cuenta la tienda, a continuación adjunto algunas capturas de las secciones y el tráfico que se mostró en ZAP con la interacción en la página (no son todas las secciones que revise, pero si unas pocas).

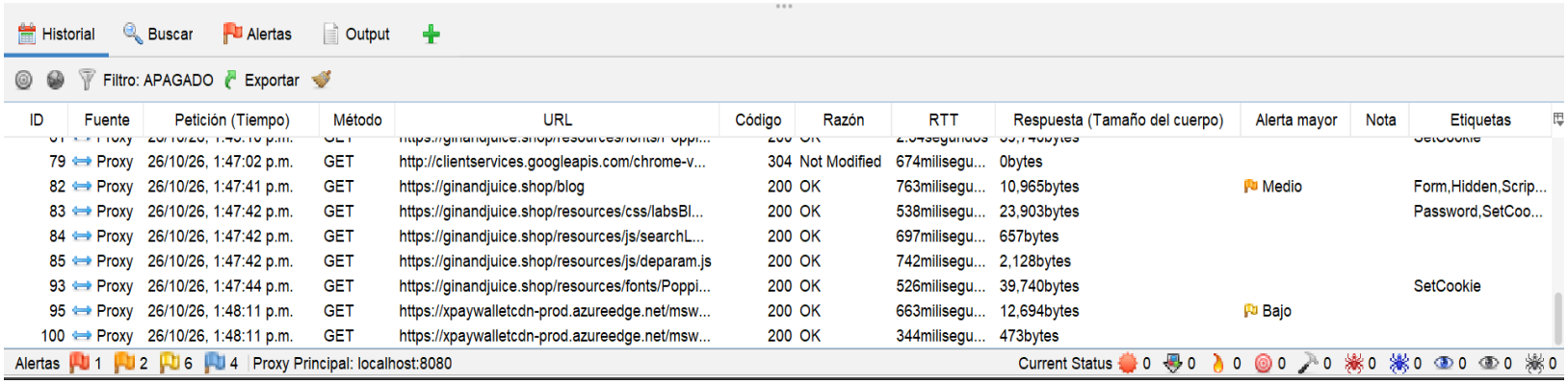
Sección de productos





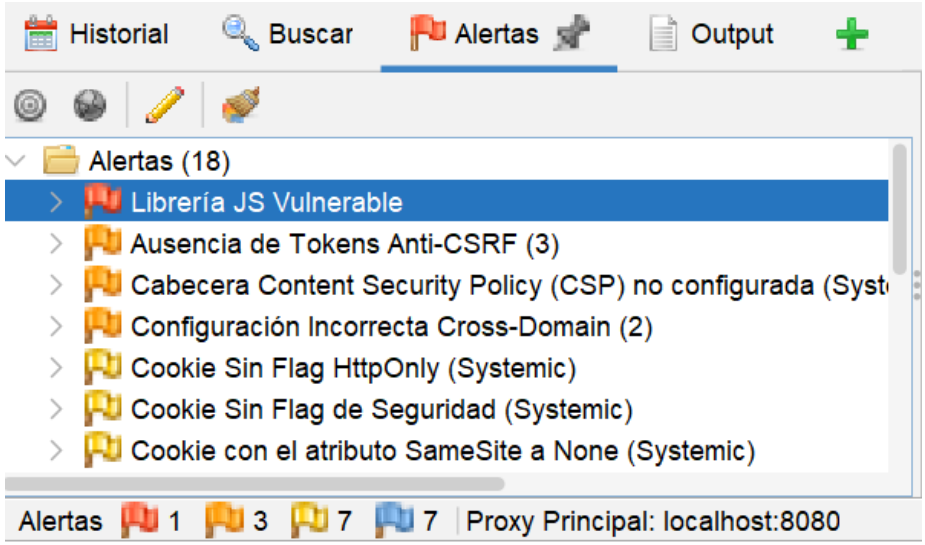
Sección de blog





3- ANALISIS DE ALERTAS

Durante el análisis del tráfico se identificaron diversas alertas de seguridad en la aplicación. A continuación, presento una tabla que resume 3 de estas alertas detectadas con diferentes niveles de riesgo durante la sesión, las cuales revise de forma general para comprender los principales hallazgos del sitio.



NIVEL DE RIESGO	NOMBRE	URL	PARAMETRO	EVIDENCIA
ALTO	*Librería JS Vulnerable*	*https://ginandjuice.shop/resources/js/angular_1-7-7.js*	-	*/* *AngularJS v1.7.7*
MEDIO	*Configuración Incorrecta Cross-Domain*	*https://ginandjuice.shop/https://ginandjuice.shop/*	-	-
BAJO	*Cookie Sin Flag HttpOnly*	*https://ginandjuice.shop/https://ginandjuice.shop/*	*AWSALB*	*Set-Cookie: AWSALB*

4- DOCUMENTACION DE HALLASGO PRINCIPAL

NOMBRE DE LA ALERTA	*Librería JS Vulnerable*
NIVEL DE RIESGO	*Alto*
URL O COMPONENTE	*https://ginandjuice.shop/resources/js/angular_1-7-7.js*
PARAMETRO AFECTADO	-
EVIDENCIA	*/* *AngularJS v1.7.7* **Librería JS Vulnerable** URL: https://ginandjuice.shop/resources/js/angular_1-7-7.js Riesgo: High Confianza: Medium Parámetro: Ataque: Evidencia: /* AngularJS v1.7.7 CWE ID: 1395 WASC ID:
DESCRIPCION TECNICA	*La biblioteca es una versión desactualizada con vulnerabilidades públicas.*
IMPACTO POTENCIAL	*La biblioteca identificada angularjs, versión 1.7.7, es vulnerable.CVE-2025-2336* *CVE-2022-25869* *CVE-2022-25844*
RECOMENDACION	*Actualice a la versión más reciente de la biblioteca afectada.*

5- CONCLUCIONES

¿QUE APRENDI?

Aprendí a familiarizarme y utilizar la plataforma de OWASP ZAP, ya que anteriormente solo la había visto en uso, pero no comprendía cómo funcionaba su entorno. Al ser una persona muy visual y de práctica, realizar la práctica me ayudó a entender cómo está distribuida la aplicación, cómo se presentan las alertas y cómo interpretar la información del tráfico HTTP. Ahora sé identificar y leer los reportes de seguridad que genera la herramienta para reconocer los principales hallazgos dentro del sitio.

¿QUÉ FUE LO MAS RETADOR?

La principal dificultad fue al inicio, cuando no podía acceder a la página por el bloqueo de seguridad. Esto lo resolví utilizando el navegador integrado que provee OWASP ZAP para facilitar la navegación y capturar el tráfico sin conflictos. Asimismo, al principio fue un reto entender qué significaban los distintos elementos en pantalla (como los métodos HTTP y las etiquetas del tráfico), pero al explorar la herramienta, moverle de aquí allá e investigar por mi cuenta, logré comprender mucho mejor cómo opera la plataforma y cómo relacionar mis acciones en la tienda con los registros de la herramienta.

¿QUE REVISARIA A CONTINUACION?

En un entorno real y con la debida autorización, me gustaría profundizar en pruebas de penetración y en la explotación controlada de las vulnerabilidades encontradas, actuando como un hacker ético. También me interesaría seguir explorando y dominando cada una de las funciones avanzadas que ofrece OWASP ZAP, ya que este ejercicio dinámico me permitió aprender lo básico y me gustaría exprimir al máximo todas las herramientas que incluye la aplicación para realizar análisis de seguridad más completos.